



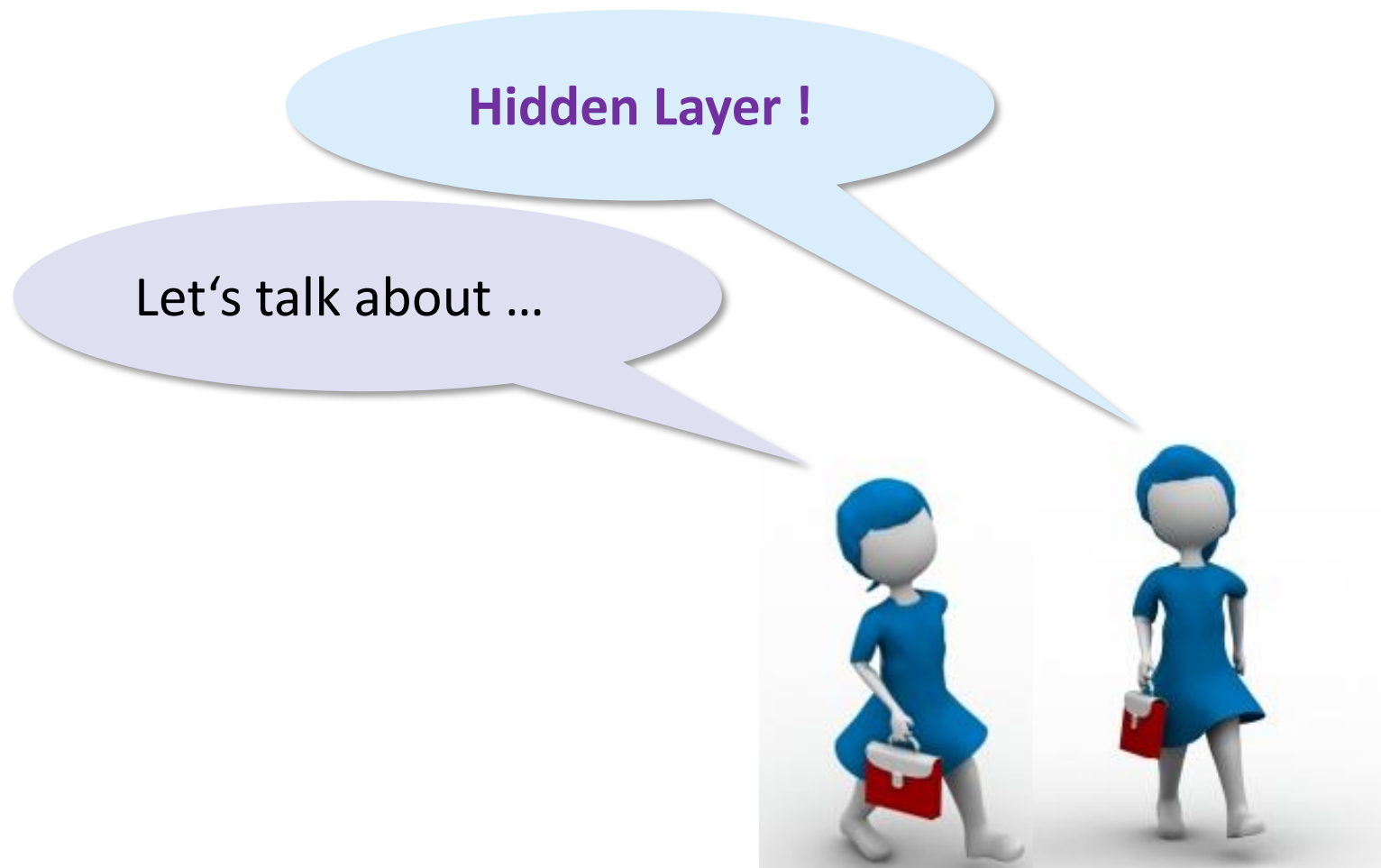
Hochschule für Angewandte Wissenschaften Hamburg
Hamburg University of Applied Sciences

Computational Intelligence

Prof. Dr.-Ing. Sabine Schumann

Fakultät Informatik und Digitale Gesellschaft

Künstliche Neuronale Netze
Architekturen



Bilder: Freedigitalphotos.net: Renjith Krishnan, ID-10082483 und ID-10086282



Architekturen

KNN können für mehrere Anwendungsklassen des Data Mining verwendet werden.

Eine Problemstellung erfordert eine spezielle Architektur und Lernverfahren, die diese Architektur unterstützen bzw. mit ihr abgestimmt sind.

Was erwartet Dich in diesem Kapitel?

Competitive Networks

Feedforward versus Rekurrent

Rekurrente KNN

Deep Learning – was ist das?

CNN

GAN



Competitive Networks

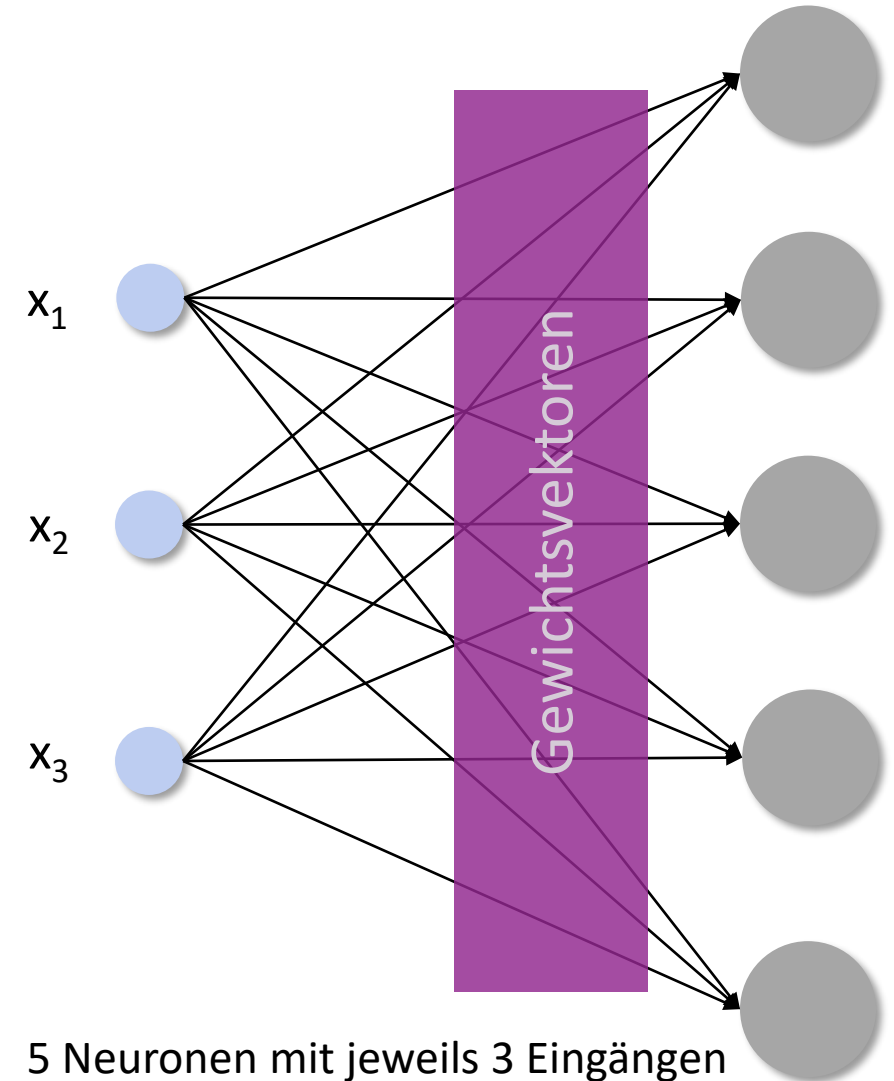
Wettbewerbs-Netze

Competitive Networks

- unüberwachtes Lernen
- Anwendungsklasse: Clustern
- es gibt nur Eingangsdaten, keine Zielvorgaben
- Strukturen / Muster sollen erkannt werden

Aufbau des Netzes / der Netze:

- Eingangsschicht ($x_1, x_2, \dots, x_n$)
- Neuronenschicht
- der angelegte Eingangsvektor X ist mit allen Neuronen verbunden und die Information wird gleichzeitig übergeben.



Arbeitsweise (1)

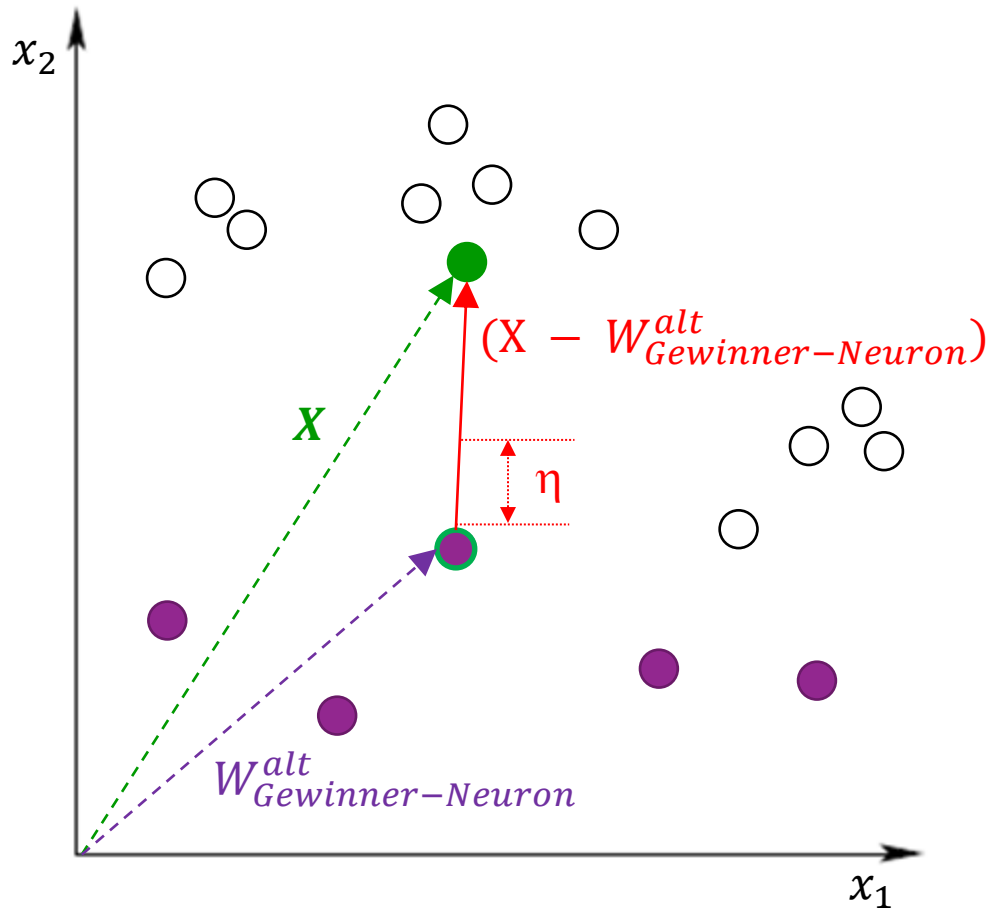
1. Wähle per Zufall einen Eingabevektor X aus dem Eingabeset aus
2. Bestimme mit Hilfe der euklidischen Distanz für jedes Neuron den Abstand der Gewichtsvektoren zur Eingabe.
Das Neuron mit dem geringsten Abstand gewinnt (Gewinner-Neuron).
3. Anpassung des Gewichtsvektors des Gewinner-Neurons (siehe nächste Folie)
4. Gehe zu Schritt 1 bis es keine wesentlichen Änderungen von Gewichtsvektoren gibt

▶ Das Verfahren konvergiert nicht zwingend!!!

▶ Das Ergebnis sind Clusterzentren – denn es gibt i.a. mehr Datenpunkte als Neuronen, so dass das Netz einen Kompromiss zwischen allen Daten finden muss.

Zur grafischen Verdeutlichung auf der folgenden Folie nehmen wir nur 2 Eingänge: x_1 und x_2 .

Arbeitsweise (2)



Legende:

- unbekannte Datenpunkte $X_i = [x_{i1}, x_{i2}]$
- Eingangsvektor
- Gewichtsvektoren der 5 Neuronen $W_i = [w_{i1}, w_{i2}]$
- Gewinner-Neuron
- η Lernrate

$$dist_{Euklid}(X, W_i) = \sqrt{\sum_j (x_j - w_{ij})^2}$$

$$W_{Gewinner-Neuron}^{neu} = W_{Gewinner-Neuron}^{alt} + \eta(X - W_{Gewinner-Neuron}^{alt})$$

Der Gewichtsvektor des Gewinner-Neurons „bewegt sich“ mit der Schrittlänge η auf den Eingangsvektor zu.

Abbildung angelehnt an Ralf Otte: *Künstliche Intelligenz für dummies*. Wiley, 1. Auflage, 2019, S. 243

Was erwartet Dich in diesem Kapitel?

Competitive Networks

➤ ein spezieller Vertreter: SOM

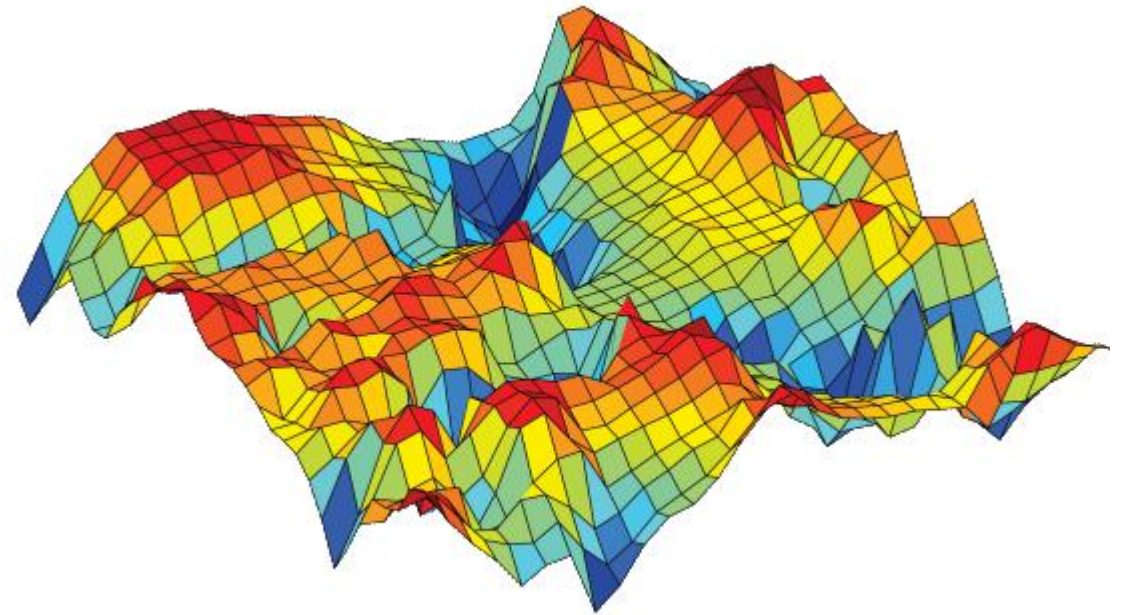
Feedforward versus Rekurrent

Rekurrente KNN

Deep Learning – was ist das?

CNN

GAN



Bildquelle: <https://www.viscovery.net/bilder/somine/UnfoldedMap.png>

Self-Organizing Maps (SOM) / Selbstorganisierende Karten

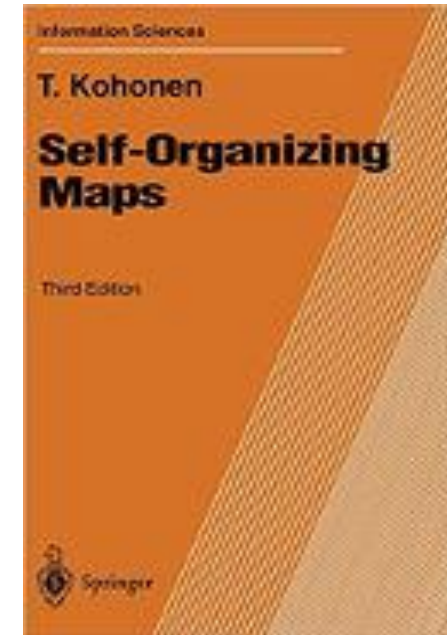
auch

Kohonen-Netzwerke, Kohonen-Map, Kohonen Feature Map, Self-Organizing Feature Maps (SOFM), k-Netz, Selbst-Organisierende Merkmalskarten, ... genannt,

wurden 1972 von Prof. Teuvo Kohonen erfunden, 1981 veröffentlicht



Teuvo Kohonen
(1934-2021)



Kohonen, Teuvo: ***Self-Organizing Maps***.
Springer Verlag, 3. Auflage, 2001

Bildquelle: <https://yle.fi/uutiset/3-12233215>

SOM - Idee

- unser Gehirn wird von der Großhirnrinde (zerebraler Kortex) (vergleiche Kapitel Künstliche Neuronale Netze, Folien 14-16) beherrscht
 - diese besteht aus einer sehr komplexen Struktur mit Milliarden von Neuronen und Hunderten von Milliarden von Synapsen
 - diese Struktur umfasst Bereiche, die für verschiedene menschliche Aktivitäten zuständig sind und mit unterschiedlichen Sinneseindrücken in Verbindung stehen.
 - jeder sensorische Input wird in einem entsprechenden Bereich der Großhirnrinde abgebildet:
-  somit ist die Großhirnrinde eine *sich selbst organisierende Rechenkarte* im menschlichen Gehirn

SOM

- werden verwendet, um eine geordnete Darstellung von mehrdimensionalen Daten zu erstellen, die die Komplexität vereinfacht und sinnvolle Beziehungen aufzeigt
- SOMs sind eine besonders robuste Form des unüberwachten Lernens
- seit den 1980ern sind sie technologische Grundlage für unzählige Anwendungen sowie Gegenstand vieler Veröffentlichungen
- nutzen wettbewerbsorientiertes Lernen (competitiv) , um Eingabedaten zu klassifizieren, ohne die erwartete Ausgabe zu kennen
- Es wird eine ästhetische Zielfunktion für eine erfolgreiche Klassifizierung verwendet.
- Nach Vorlage eines Eingabemusters bewertet das Netz, welcher seiner Knoten am ehesten mit dieser Eingabe übereinstimmt, und passt sie dann zusammen mit ihren Nachbarknoten an, um die Übereinstimmung weiter zu verbessern.
- Die einzugebenden Daten müssen numerisch kodiert sein.

SOM – Methode

- kann als nichtparametrisches Regressionsverfahren betrachtet werden, das mehrdimensionale Datenräume in niedrigere dimensionale Abstraktionen umwandelt
- ähnlich wie eine Regressionsebene eine Abstraktion der ursprünglichen Daten darstellt, erzeugt eine SOM eine Darstellung der Datenverteilung, allerdings mit dem wichtigen Unterschied, dass diese Darstellung nicht linear ist
- Knoten auf dem Gitter sind sogenannte *Referenzvektoren* zugeordnet, die auf bestimmte Regionen im ursprünglichen Datenraum zeigen
- ausgehend von numerischen, multivariaten Datensätzen passen sich diese *Referenzvektoren* auf dem Gitter allmählich an die intrinsische Form der Datenverteilung an, wobei die Referenzvektoren benachbarter Knoten auf benachbarte Regionen im Datenraum zeigen
- so spiegelt die Anordnung auf dem Gitter die Nachbarschaft innerhalb der Daten wider, so dass die Merkmale der Datenverteilung direkt aus der entstehenden Landschaft auf dem Gitter abgelesen werden können

SOM – Aufbau

SOMs bestehen aus 2 Schichten von Neuronen

- eindimensionale Eingabe-Schicht repräsentiert die Daten
- Kohonen-Schicht (auch Karten-Schicht) Verarbeitungs- und Ausgabeschicht

Die Eingabe-Schicht ist vollständig mit der Kohonen-Schicht verbunden.

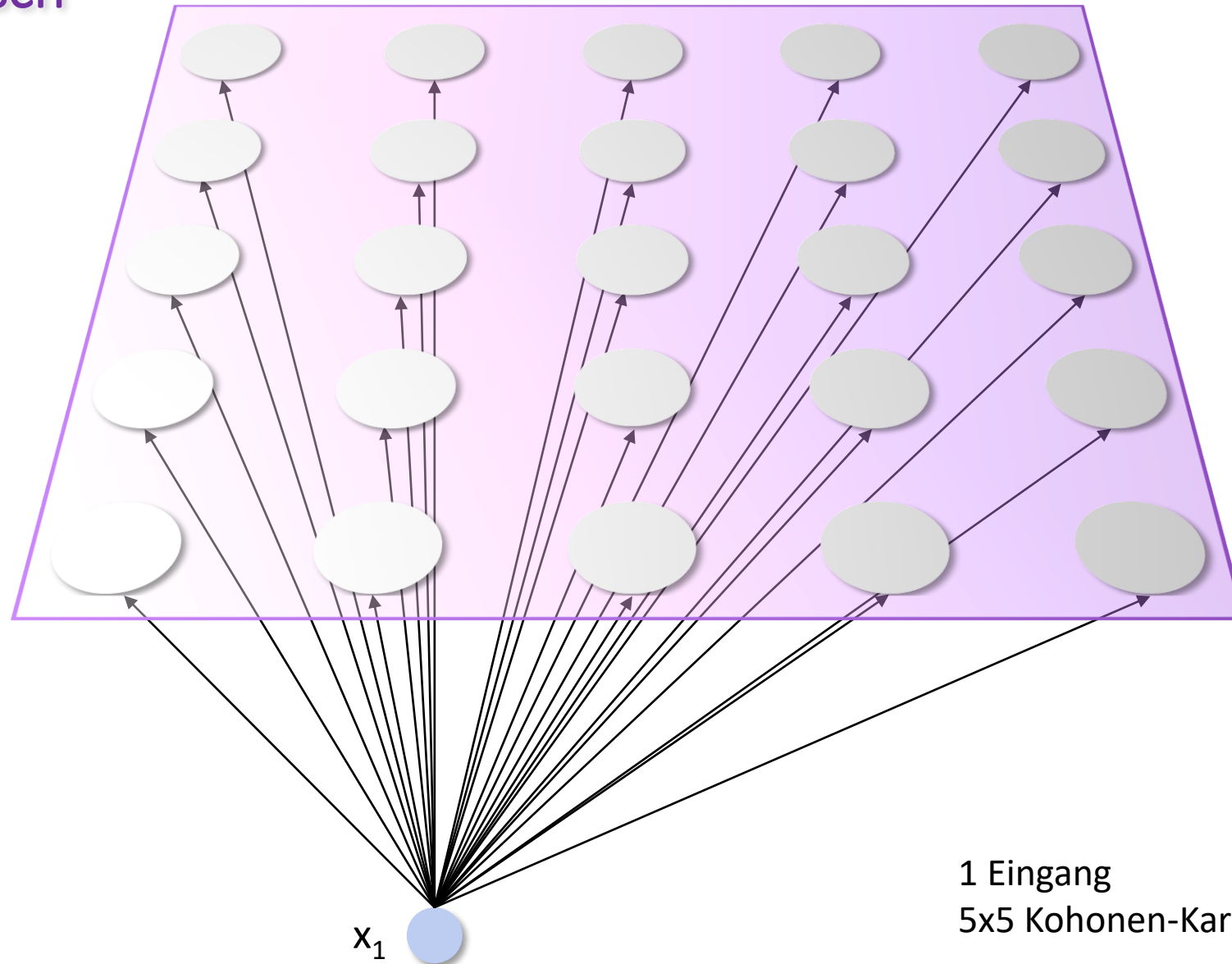
Neuronen der Kohonen-Schicht sind implizit untereinander verbunden

- keine trainierbaren Gewichte
- geometrischer Abstand zweier Neuronen wird als Gewicht zwischen den Neuronen interpretiert
- die Nachbarschaft (nah oder fern) wird beim Trainingsprozess berücksichtigt

Die Anzahl der Neuronen der Eingabeschicht entspricht der Anzahl der Werte des Musters im Wertebereich $[0,1]$ (numerische Codierung).

Anzahl der Neuronen der Kohonen-Schicht kann nur experimentell bestimmt werden. Eine 10x10 Matrix hat sich als guter Startwert herausgestellt.

SOM grafisch



1 Eingang
5x5 Kohonen-Karte = 25 Neuronen

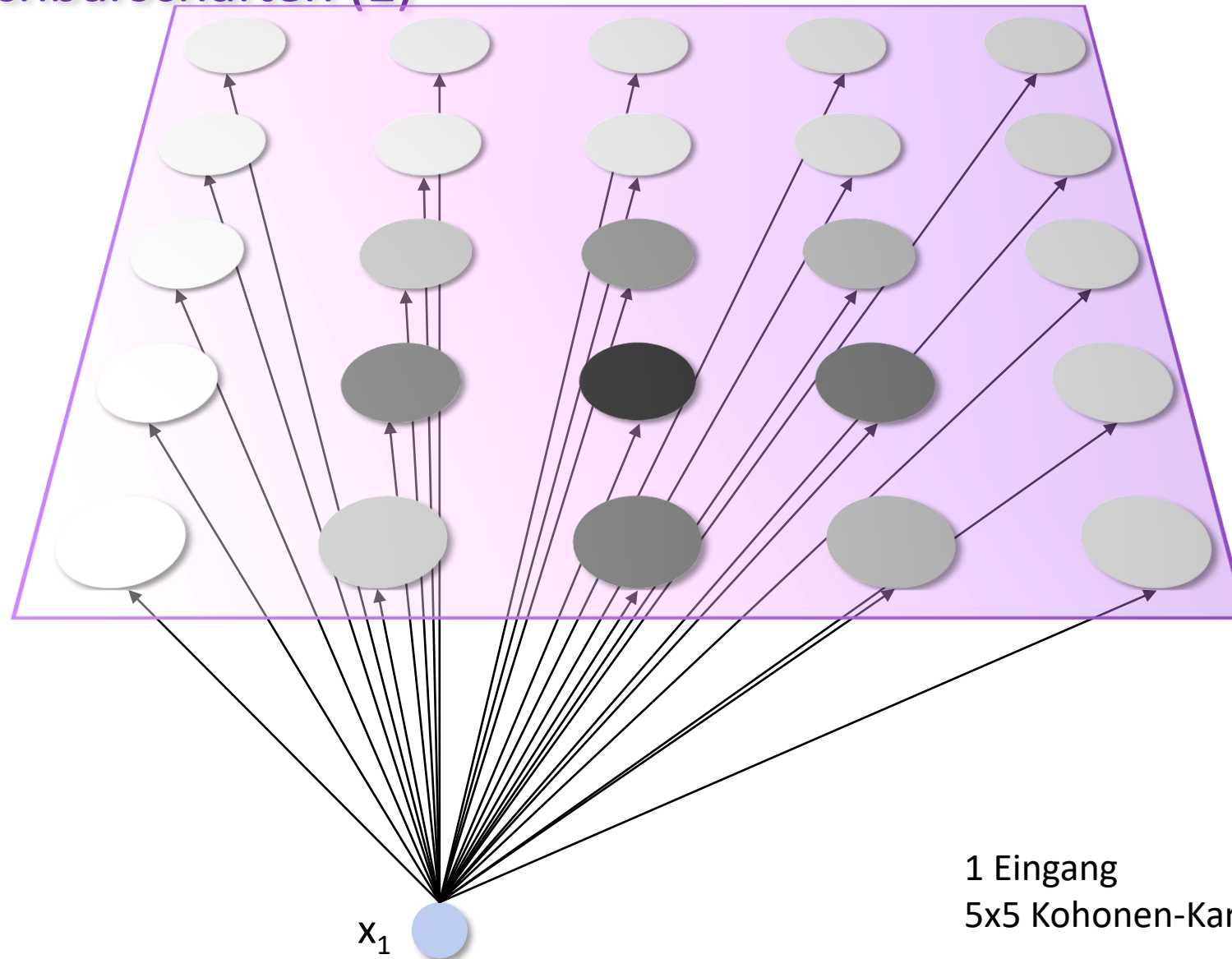
SOM – Lernen

- die Neuronen konkurrieren untereinander um die Aktivierung, aber nur ein einziges Ausgangsneuron kann zu jedem Zeitpunkt aktiv sein
- das Ausgangs-Neuron, das den Wettbewerb gewinnt, wird als "**winner-takes-all**"-Neuron (Gewinner-Neuron) bezeichnet
- das Training beginnt mit der Nachbarschaft des Gewinner-Neurons, im weiteren Verlauf des Trainings nimmt die Größe der Nachbarschaft allmählich ab

Wir brauchen also noch zusätzlich eine Berechnung der Nachbarschaft!

▶ Ohne die Nachbarschaftsbeziehung in der Berechnung hätten wir ein klassisches **Competitive-Network**.

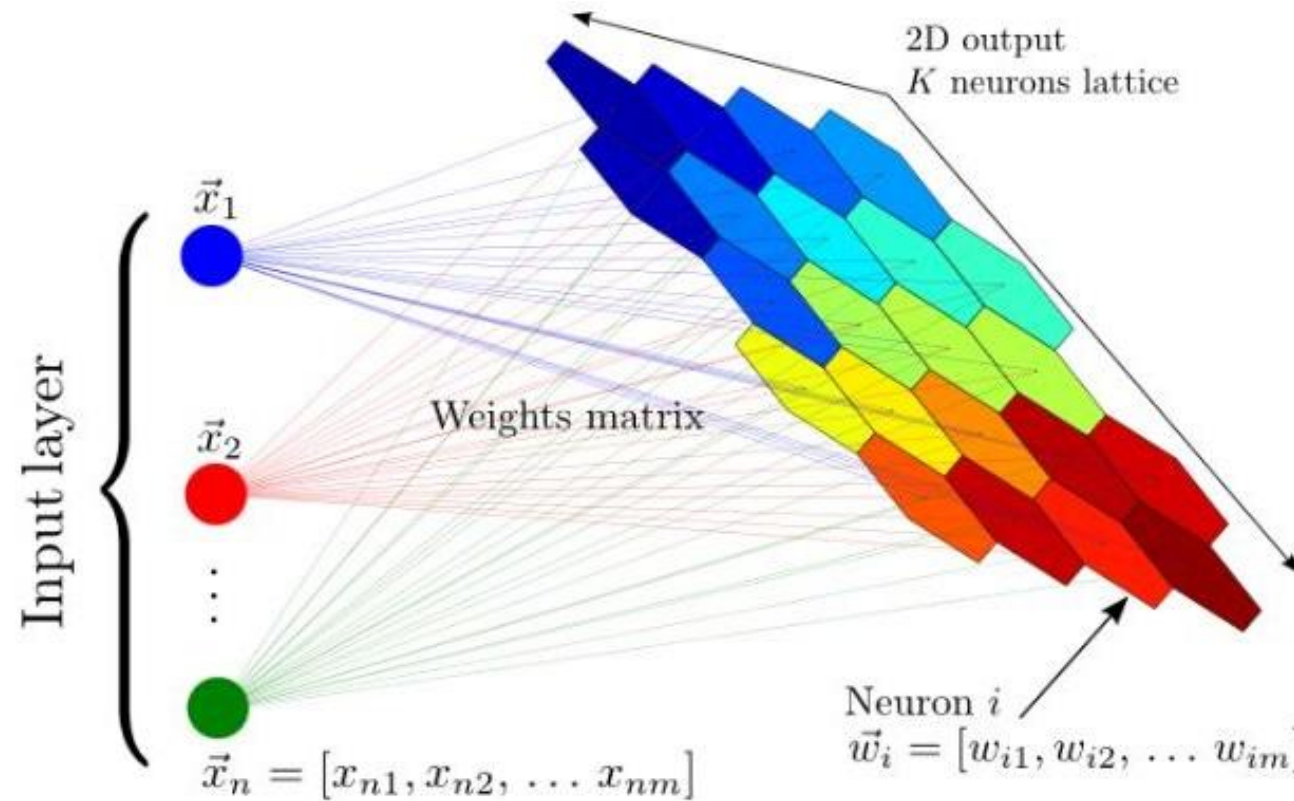
SOM – Nachbarschaften (1)



1 Eingang
5x5 Kohonen-Karte = 25 Neuronen

SOM – Nachbarschaften (2)

Für Data-Mining-Zwecke hat es sich eingebürgert, die SOM durch ein zweidimensionales hexagonales Gitter zu approximieren.



Bildquelle: arxiv.org/pdf/1312.5753.pdf

SOM – Nachbarschaften (3)

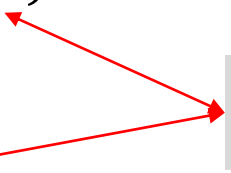
- Seien die Eingangsdaten in einem n-dimensionalen Datenraum über den Vektor $X = (x_1, x_2, \dots, x_i, \dots, x_n)$ beschrieben.
- Der Gewichtsvektor W_g eines Neurons g besteht aus n Komponenten $W_g = (w_{g1}, w_{g2}, \dots, w_{gi}, \dots, w_{gn})$.
- Das Gewinner-Neuron wird durch $W_{g'}$ beschrieben.
- Die Anpassung des Gewichtsvektors erfolgt nicht nur für das Gewinner-Neuron g' , sondern auch für dessen Nachbarn g auf der Karte in Abhängigkeit einer räumliche Funktion $h_{gg'}$.

$$W_g^{neu} = W_g^{alt} + \eta h_{gg'}(X - W_g^{alt})$$

mit bspw. Gaußscher Glockenfunktion:

$$h_{gg'} = \frac{1}{d\sqrt{2\pi}} e^{\frac{-(W_g - W_{g'})^2}{2d^2}}$$

Die Variable d bestimmt die räumliche Ausdehnung der Gaußglocke und wird im Laufe des Lernens verändert.


$$dist_{Euklid}(X, W_i) = \sqrt{\sum_j (x_j - w_{ij})^2}$$

Was erwartet Dich in diesem Kapitel?

Competitive Networks

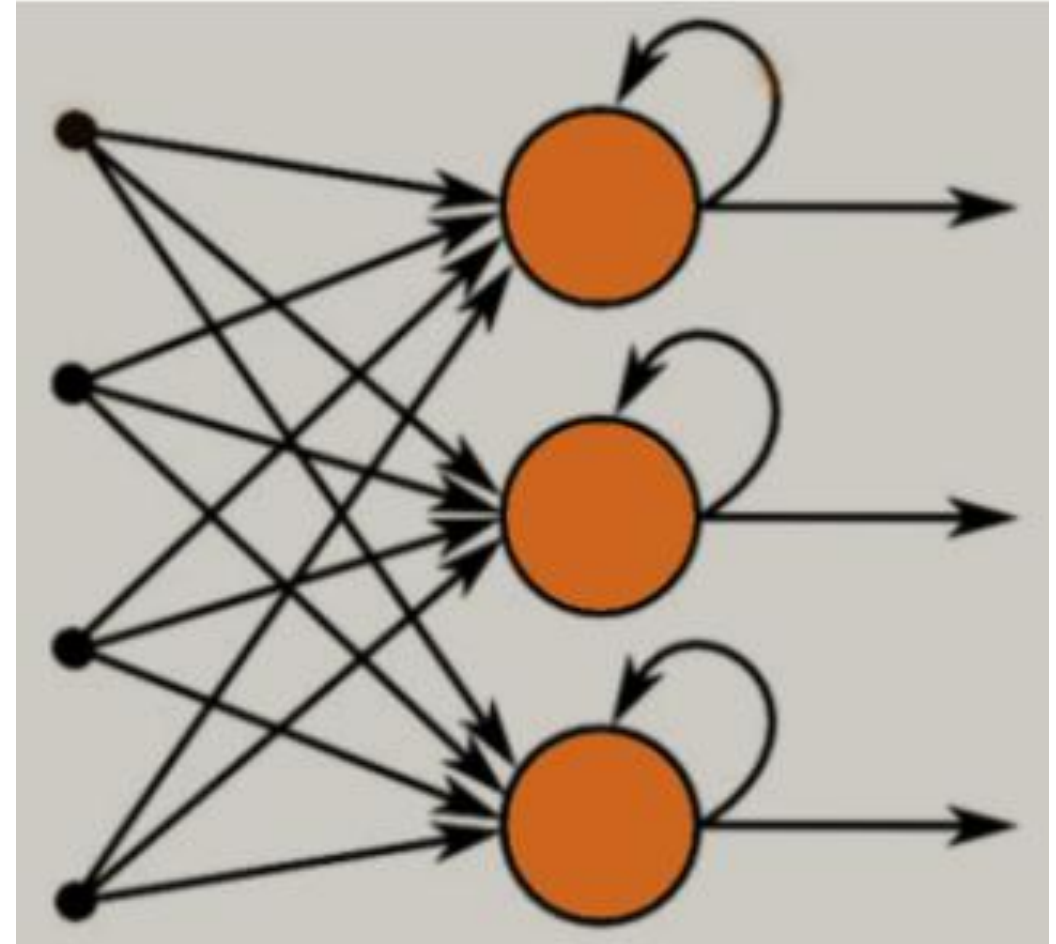
Feedforward versus Rekurrent

Rekurrente KNN

Deep Learning – was ist das?

CNN

GAN



Bildquelle:

<https://k21academy.com/datascience/machine-learning/recurrent-neural-networks/>

RNN

~~Rhein Nahe Nahverkehrsverbund~~



Rekurrente Neuronale Netzwerke

Bildquelle: <https://www.rnn.info/>

Feedforward und Rekurrente Netze (Wiederholung)

Feedforward KNNs

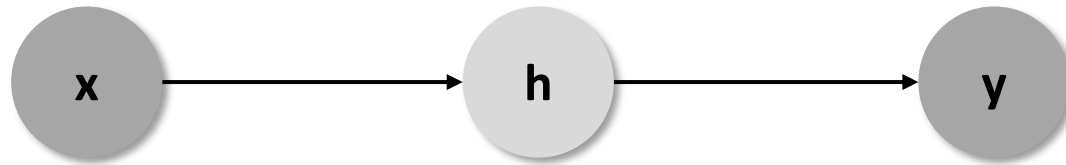
- **Einschichtiges feedforward-Modell**
 - besteht nur aus einer Eingabe- und Ausgabe-Schicht
- **Multilayer feedforward-Modell**
 - zwischen Eingabe und Ausgabe-Layer sind verdeckte (hidden) Neuronen(Layer), deren Ausgabe außerhalb des Netzes nicht sichtbar ist,
 - ebenenweise verbunden,
 - nur Verbindungen zwischen benachbarten Layern, nur vorwärts gerichtet
- **Mehrschichtiges feedforward-Modell mit Abkürzungen**
 - Verbindungen auch zwischen nicht benachbarten Layern (Shortcuts)

Rekurrente KNNs

- **direkte Rückkopplung (direct feedback)**
 - es gibt rückwärtsgerichtete (rekurrente, feedback loops) Kanten zum selben Neuron
- **indirekte Kopplung (indirect feedback)**
 - zwischen Eingabe- und Ausgabe-Schicht ist meist ein mehrschichtiges Netz mit verdeckten (hidden) Neuronen(Layern),
 - Rückkopplung über mehrere Layer in der eigenen Ebene
- **laterale Rückkopplung (winner-takes-all)**
 - ohne direkte Rückkopplung innerhalb eines Layers,
 - Rückmeldung der Information an Neuronen des eigenen Layers

Rekurrent versus Feedforward

Feedforward Netz:

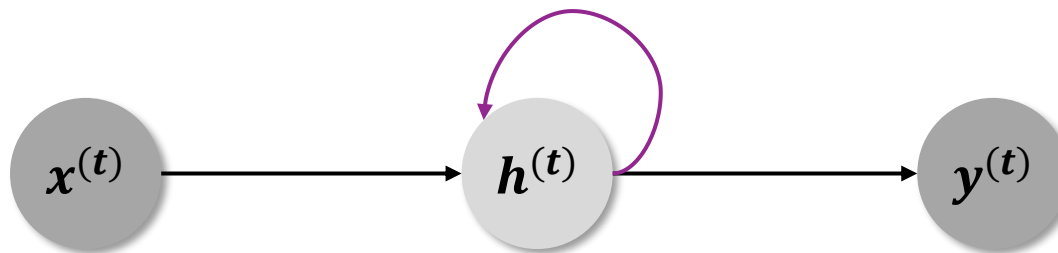


Der Einfachheit halber:

x – Eingabeschicht
 y – Ausgabeschicht
 h – verdeckte Schicht (hidden layer)

Es KANN sich dabei um Vektoren handeln, die viele Neuronen enthalten.

(einschichtiges) Rekurrentes Netz mit direkter Rückkopplung:



Was erwartet Dich in diesem Kapitel?

Competitive Networks

Feedforward versus Rekurrent

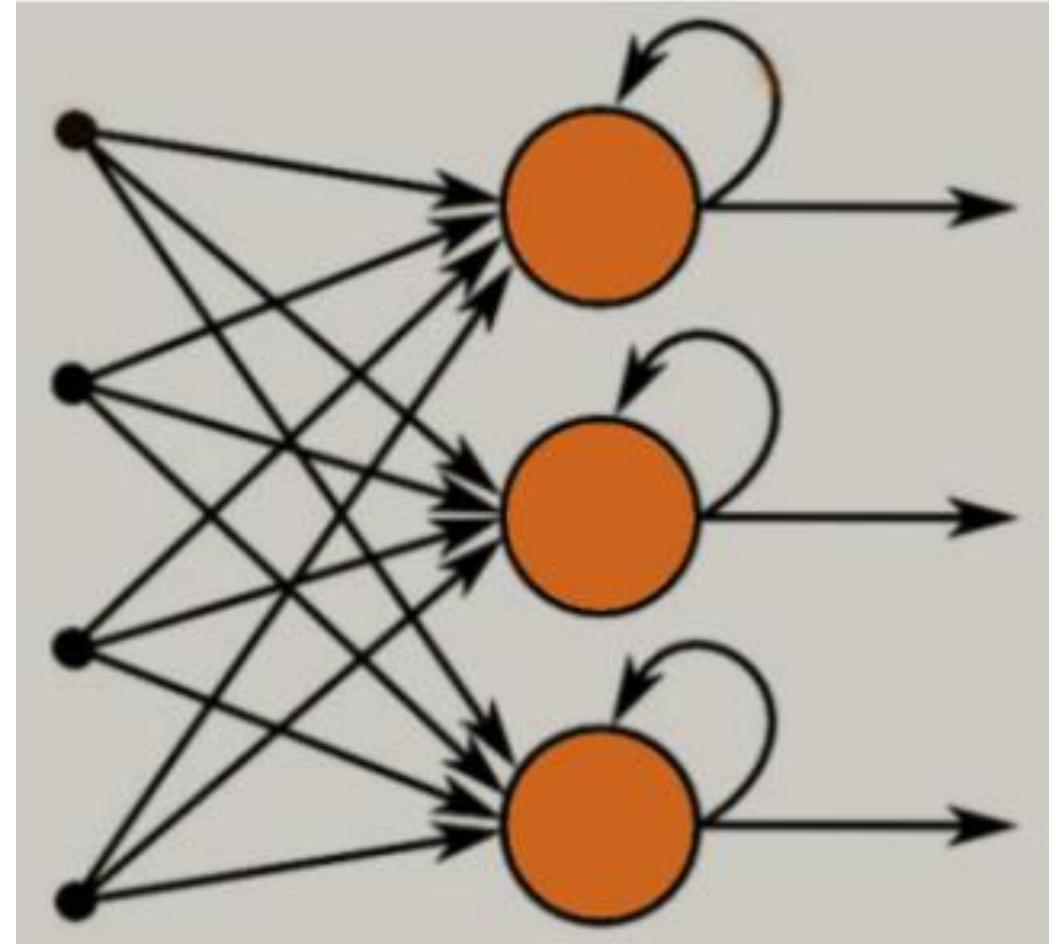
Rekurrente KNN

- Informationsfluss
- Backpropagation
- LSTM
- GRU

Deep Learning – was ist das?

CNN

GAN



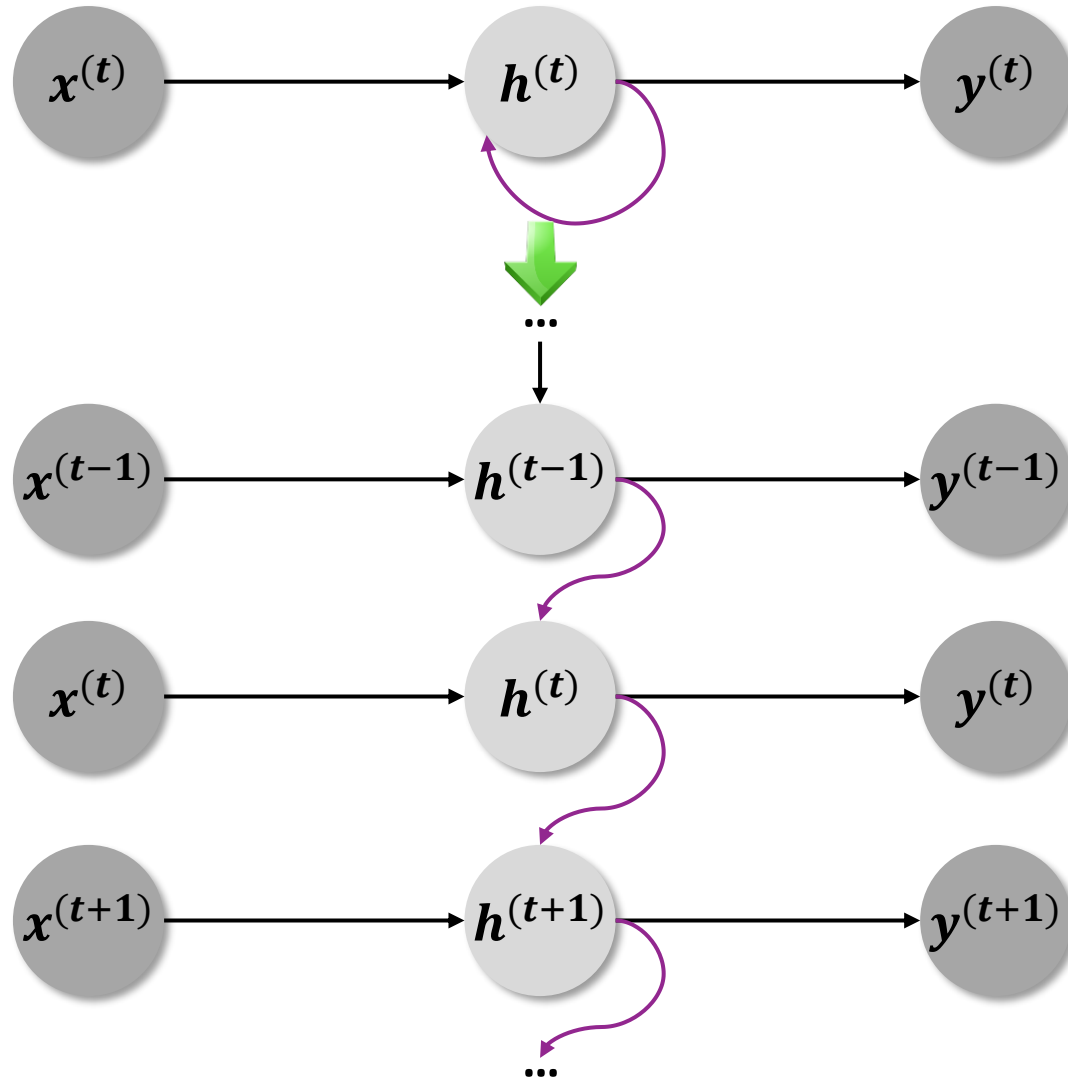
Bildquelle:

<https://k21academy.com/datascience/machine-learning/recurrent-neural-networks/>

Informationsfluss im RNN

- Jedes Neuron der Eingabeschicht erhält nur eine Eingabe
- Jedes verdeckte Neuron erhält 2 verschiedene Eingaben:
 - Voraktivierung der Eingabeschicht und
 - Aktivierung derselben verdeckten Schicht des vorherigen Schrittes zum Zeitpunkt $t-1$
- Zeitpunkt $t=0$
 - Alle Neuronen werden mit Nullen oder kleinen zufälligen Werten initialisiert
- Zeitschritt $t>0$
 - Eingabe der $x^{(t)}$ und die letzten Werte der hidden Layer zum Zeitpunkt $t-1$: h^{t-1}

Einschichtiges RNN mit direkter Rückkopplung



Der Einfachheit halber:

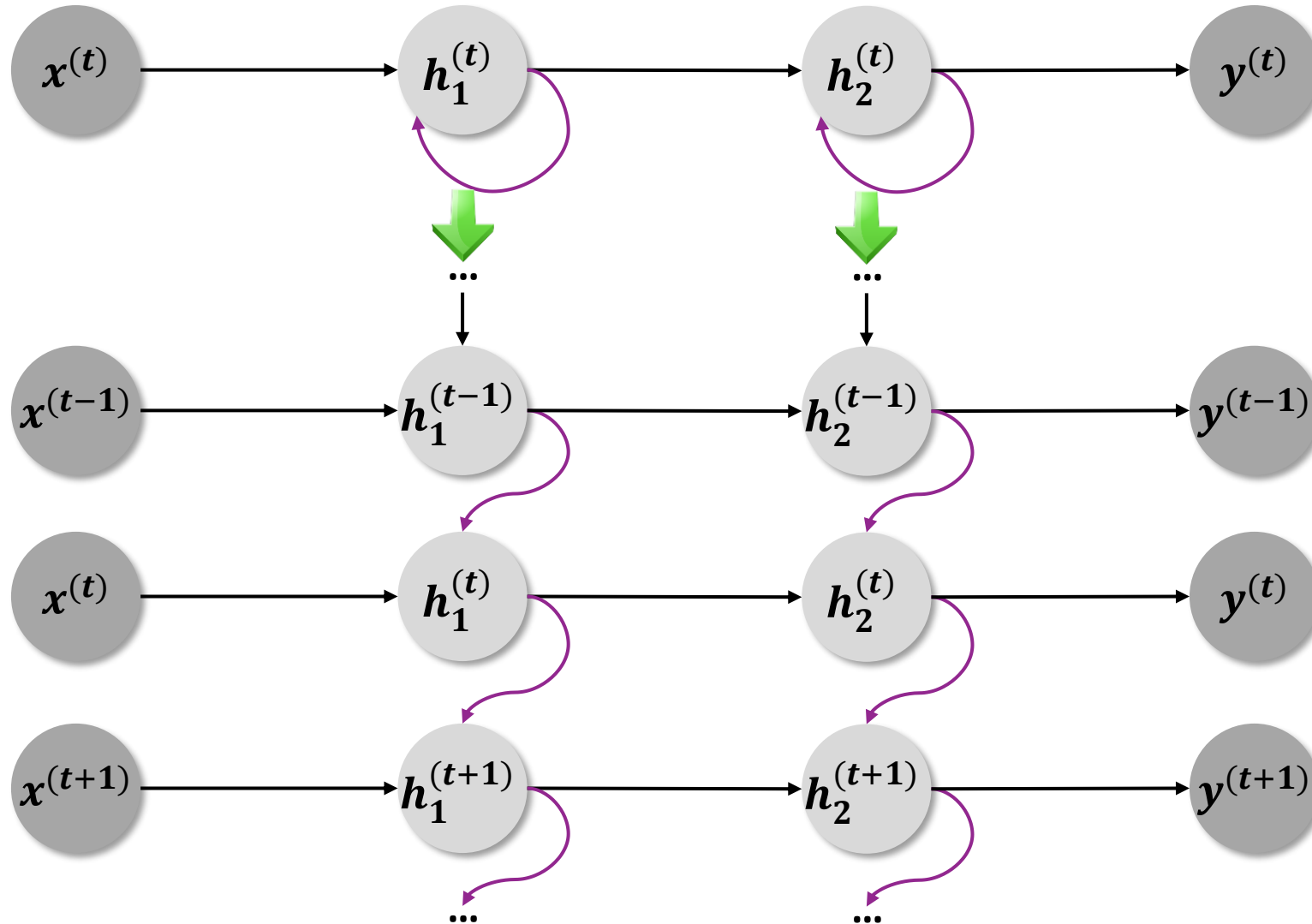
x – Eingabeschicht

y – Ausgabeschicht

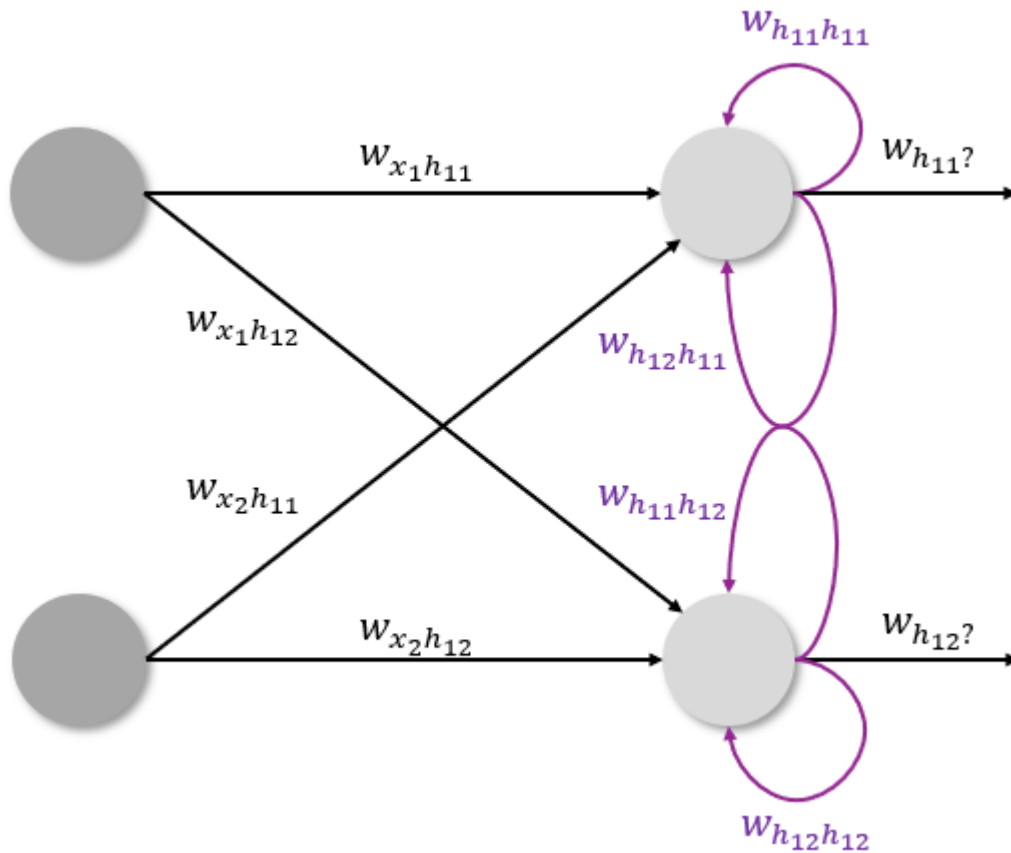
h – verdeckte Schicht (hidden layer)

Es KANN sich dabei um Vektoren handeln,
die viele Neuronen enthalten.

Mehrschichtiges RNN mit direkter Rückkopplung



Aktivierung eines RNN berechnen (am Beispiel einschichtiges RNN)



$$W_{xh} \quad x^{(t)} \quad + \quad W_{hh} \quad h^{(t-1)} \quad + \quad b_h$$

$$h^{(t)} = \Phi_h(W_{xh}x^{(t)} + W_{hh}h^{(t-1)} + b_h) \quad (1.1)$$

Endgültige Ausgabe:

$$y^{(t)} = \Phi_y(W_{hy}h^{(t)} + b_y) \quad (1.2)$$

Trainieren von RNN durch Backpropagation Through Time – BPTT

Backpropagation Through Time - BPTT

- 1990 vorgestellt in Werbos, Paul:
Backpropagation through time: what it does and how to do it.
Proceedings of the IEEE 78(19), November 1990, S. 1550-1560

Herleitung des Gradienten etwas kompliziert

- Idee: Gesamtverlust L als Summe der Verlustfunktionen $t = 1$ bis $t = T$ ergibt: $L = \sum_{t=1}^T L^{(t)}$
- Verlust zum Zeitpunkt t ist abhängig von den verdeckten Neuronen vorhergehender Zeitschritte

Gradient:

$$\frac{\delta L^{(t)}}{\delta W_{hh}} = \frac{\delta L^{(t)}}{\delta y^{(t)}} \times \frac{\delta y^{(t)}}{\delta h^{(t)}} \times \left(\sum_{k=1}^t \frac{\delta h^{(t)}}{\delta h^{(k)}} \times \frac{\delta h^{(k)}}{\delta W_{hh}} \right) \quad (1.3)$$

mit $\frac{\delta h^{(t)}}{\delta h^{(k)}}$ als Produkt der aufeinanderfolgenden Zeitschritte:

$$\frac{\delta h^{(t)}}{\delta h^{(k)}} = \prod_{i=k}^t \frac{\delta h^{(i)}}{\delta h^{(i-1)}} \quad (1.4)$$

nach [Ra 2018], S. 536

Probleme bei Erkennung weitreichender Interaktionen (1)

Problem des verschwindenden oder explosionsartig wachsenden Gradienten bei BPTT:

- der Faktor $\frac{\delta h^{(t)}}{\delta h^{(k)}}$ besteht aus $t - k$ Faktoren
- die $(t - k)$ -fache Multiplikation mit der Gewichtung w führt zu einem Faktor w^{t-k}
- w^{t-k} wird
 - sehr klein, wenn $|w| < 1$ und $t - k$ groß
 - sehr groß, wenn $|w| > 1$ und $t - k$ groß

Achtung:
 $t - k$ bezieht sich auf weitreichende Abhängigkeiten!

Veranschaulichung des Effektes (mit einer einfachen reellen Zahl):

Iterationen	10	100	1000
Gewicht			
$ w = 0,99$	$0,99^{10} = \mathbf{0,904}$	$0,99^{100} = \mathbf{0,366}$	$0,99^{1000} = \mathbf{0,00004}$
$ w = 1,01$	$1,01^{10} = \mathbf{1,105}$	$1,01^{100} = \mathbf{2,705}$	$1,01^{1000} = \mathbf{20.959,1}$

Probleme bei Erkennung weitreichender Interaktionen (2)

Für Interessierte sei folgender Artikel empfohlen:

Pascanu, Razvan; Mikolov, Tomas; Bengio, Yoshua:
On the difficulty of training Recurrent Neural Networks.
arXiv.1211.5063v2 [cs.LG] 16. Februar 2013

3 Lösungen:

- Naive Lösung: Sicherstellen, dass $|w| = 1$
- Truncated Backpropagation-Through-Time (TBPTT)
- Long Short-Term Memory (LSTM)

Truncated Backpropagation-Through-Time – TBPTT

Idee von TBPTT:

Der Gradient wird oberhalb eines Grenzwertes abgeschnitten



explosionsartiges Anwachsen wird verhindert



Anzahl der Schritte wird begrenzt, in denen der Gradient zur ordnungsgemäßen Aktualisierung der Gewichte verwendet werden kann

Long Short-Term Memory – LSTM (1)

LSTM sind spezielle RNN:

Long Short-Term Memory (langes Kurzzeitgedächtnis)

- Rekurrentes Netz
- Selbstlernender wachsender Kurzzeitspeicher

Verwendung

- Verarbeitung von Sprache (Phoneme – Laute)
- beim autonomen Fahren
- Alexa und Siri
- ...

Long Short-Term Memory – LSTM (2)

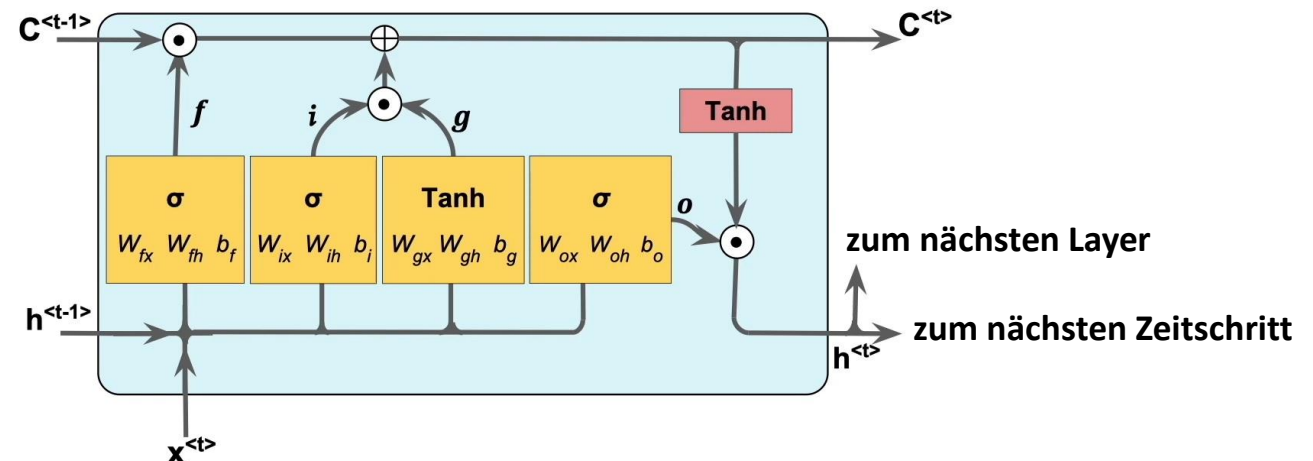
1997 von Hochreiter und Schmidhuber entwickelt

Hochreiter, Sepp; Schmidhuber, Johannes: **Long Short-term Memory**.
Neural Computation, 9(8), Dezember 1997, S. 1735-1780

Idee: Problem des verschwindenden Gradienten lösen

- Bausteine eines LSTM sind *Speicherzellen*, die quasi einen verdeckten Layer abbilden
- Jede Speicherzelle hat eine rekurrente Kante mit der gewünschten Wichtung $w=1$
- Diese den rekurrenten Kanten zugeordneten Werte werden als *Zellzustand* bezeichnet

Struktur einer (heutigen) LSTM-Zelle:
(aus [Ra 2018], S. 538)



Long Short-Term Memory – LSTM (3)

Es gibt verschiedene LSTM-Varianten. Bei Interesse ist folgender Beitrag lesenswert:

Jozefowicz, Rafal; Zaremba, Wojciech; Sutskever, Ilya:

An empirical exploration of recurrent network architectures.

ICML'15: Proceedings of the 32nd International Conference on International Conference on Machine Learning, Volume 37, Juli 2015, S. 2342–2350

Gated Recurrent Unit – GRU

- 2014 vorgestellt
- Einfachere Architektur als bei LSTM
- im allgemeinen dadurch effizienter als LSTMs

Bei Interesse sei der folgende Beitrag empfohlen:

Chung, Junyoung; Gülçehre, Çağlar; Cho, KyungHyun; Bengio, Yoshua:
Empirical evaluation of Gated Recurrent Neural Network on Sequence Modeling.
arXiv:1412.3555v1 [cs.NE], Dezember 2014

Was erwartet Dich in diesem Kapitel?

Competitive Networks

Feedforward versus Rekurrent

Rekurrente KNN

Deep Learning – was ist das?

- Cybenko-Theorem
- GPU

CNN

GAN

Deep Learning – warum?

Beispiel Bilderkennung:

- Mensch erkennt ein Objekt in einem Bild egal,
 - an welcher Position es sich befindet (translationsinvariant)
 - ob es gedreht wurde (rotationsinvariant)
 - ...
- RNN
 - kleinste translatorische Verschiebungen führen bereits zu Fehlklassifikationen

 Notwendigkeit der Weiterentwicklung

Translation Invariance



Rotation/Viewpoint Invariance



Size Invariance



Illumination Invariance



Deep Learning – was ist das?

- Mathematische Basis wurde bereits in den 1980ern gelegt

Backpropagation-Algorithmus

- Technische Basis um die Jahrtausendwende

GPU-Computing

- Cybenko Theorem (1989)
Mit einem einzigen Hidden Layer bei Feed-Forward-Netzen kann jede stetige Funktion approximiert werden.



„If σ is any continuous sigmoidal function, then the family of functions parametrized by a one layer neural network is dense, with respect to the supremum norm, in the space of continuous functions on the unit cube.“

George Cybenko

Cybenko, George: *Approximation by superpositions of a sigmoidal function*.
Mathematics of Control, Signals, and Systems. 2, 1989, S. 303–314. <https://link.springer.com/article/10.1007%2FBF02551274>

Cybenko Theorem und Kurt Hornik



Sinngemäß:

Ein Multi-Layer-Perceptron (Multi-Layer-Netz) mit mindestens einem Hidden Layer und einer endlichen Anzahl von Neuronen mit einer sigmoiden Aktivierungsfunktion kann (unter einfachen Annahmen) jede stetige Funktion approximieren.

Kurt Hornik (1991):

Es kommt gar nicht auf die spezifische Aktivierungsfunktion an, es ist die **Architektur des Netzes** selbst, dass das Netz zu einem „universellen“ Approximatoren macht.

Hornik, Kurt:

Approximation capabilities of multilayer feedforward networks.

Neural Networks. 4, 1991, S. 251–257

Grafikprozessor – GPU

Problem mit CPU

- Komplexe Aufgaben sind nicht mehr „schnell“ machbar

Lösungsidee GPU

- Grafikkarte = kleiner Rechnerverbund
- Grafikkarten sind preiswerter als CPU (etwa 70% preiswerter) und leistungsfähiger

➡ Schreiben von GPU-Code nicht so trivial

- CUDA,
- OpenCL,
- TensorFlow

Was erwartet Dich in diesem Kapitel?

Competitive Networks

Feedforward versus Rekurrent

Rekurrente KNN

Deep Learning – was ist das?

CNN

- Camera obscura
- Faltung in der Mathematik

GAN

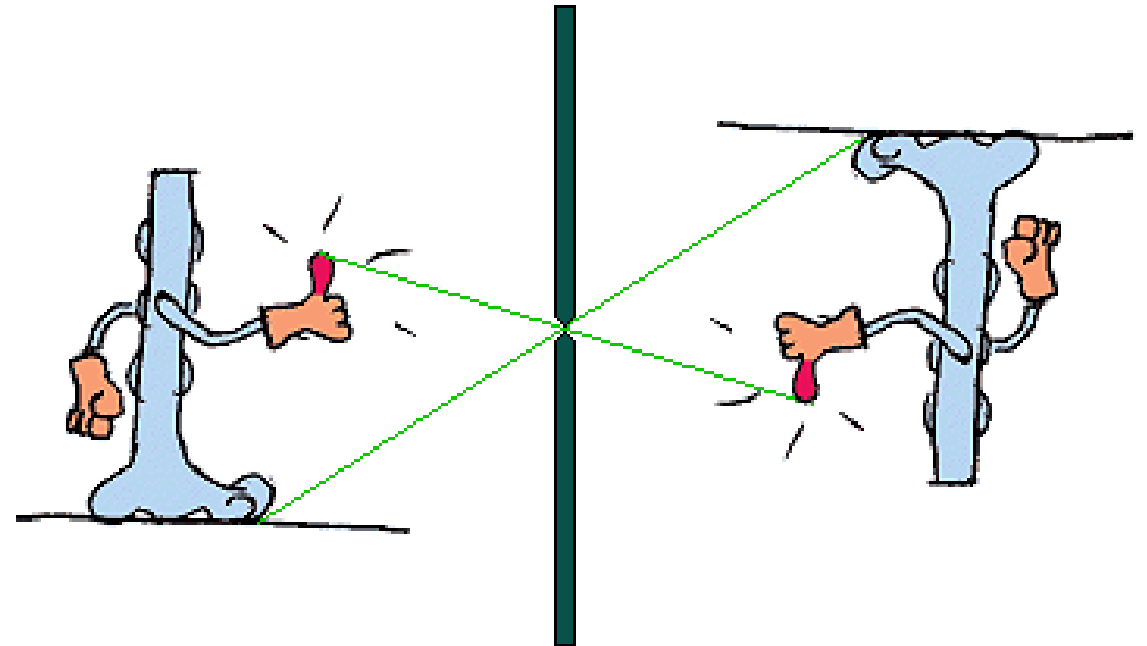


Bild: http://www.statistics4u.info/fundstat_germ/cc_convolution.html

Grundlagen

Probleme beim KNN mit Bildern

- Bei Auflösung
- Räumliche Anordnung und Verschiebung ([Invarianzen](#))

Idee:

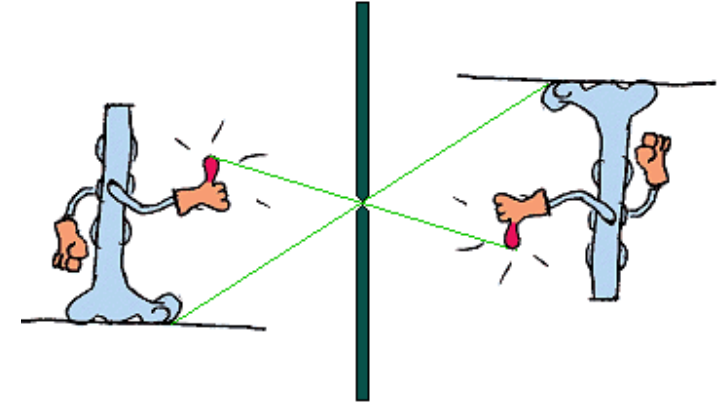
Dimensionsreduktion mittels Faltung (convolutional operation)
(Mathematik: Funktionalanalysis)

Camera obscura (1)

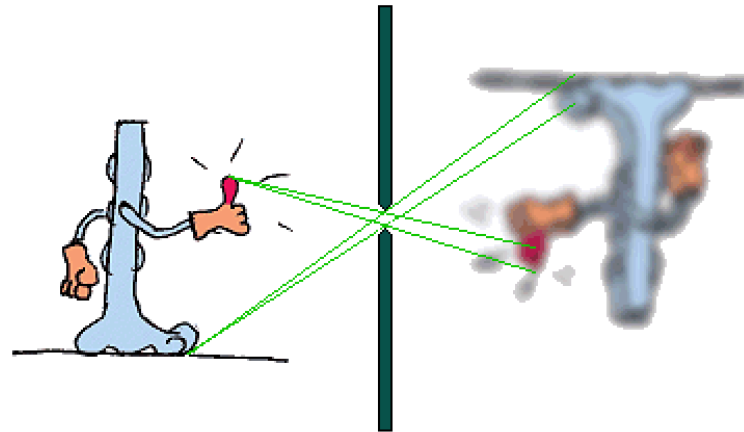


Aristoteles
(384 – 322 v. Chr.)

- Bereits Aristoteles beobachtete, dass ein Bild auf dem Kopf steht, wenn es durch ein kleines Loch auf eine gegenüberliegende Fläche projiziert wird. Das Loch ist so klein, dass es nur einen **einzigsten Lichtstrahl** durchlässt!
- Grundprinzip der Fotografie



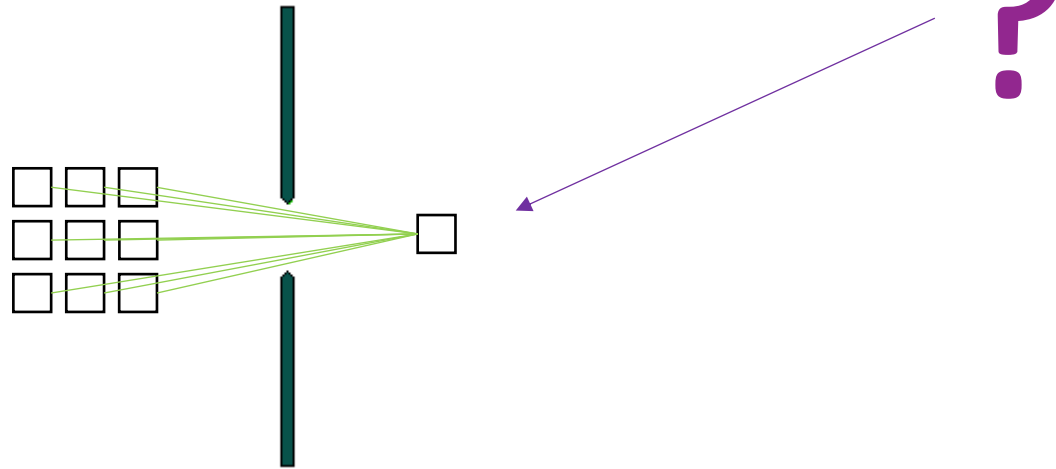
Ist das Loch größer, dann wird das projizierte Bild verunschärft.



Bildquellen: https://upload.wikimedia.org/wikipedia/commons/thumb/a/ae/Aristotle_Altemps_Inv8575.jpg/440px-Aristotle_Altemps_Inv8575.jpg
http://www.statistics4u.info/fundstat_germ/cc_convolution.html

Camera obscura (2)

- berechnet werden sollen die Bildpunkte auf der „rechten“ Seite
- Ein Punkt auf der rechten Seite umfasst (weil das Loch nicht optimal klein ist) mehrere Quellenpunkte.



- Im rechten Pixel kommt die Summe der Lichtstrahlen der (in unserem Beispiel) neun Quellenpunkte an – jedoch nicht unbedingt gleich stark, sondern gewichtet.
- Man kann sich nun vorstellen, dass ein Filter (in unserem Beispiel 3×3) über das Quellbild wandert, und ein verrauschtes, eventuell etwas verkleinertes Abbild erzeugt.

die (mathematische) Faltung (1)

Zur Vereinfachung gehen wir nachfolgend davon aus, dass die Bilder und der Filter jeweils quadratisch sind. Die Dimensionen bezeichnen wir mit $\dim$.

Alle Pixel des Abbildes Abb ergeben sich dann aus der folgenden Formel, wobei Abb_{ij} das Pixel an Position Zeile i und Spalte j ist:

$$Abb_{ij} = \sum_{m=1}^{\dim(Filter)} \sum_{n=1}^{\dim(Filter)} Quelle_{i+m-1,j+n-1} * Filter_{mn} \quad (1.5)$$

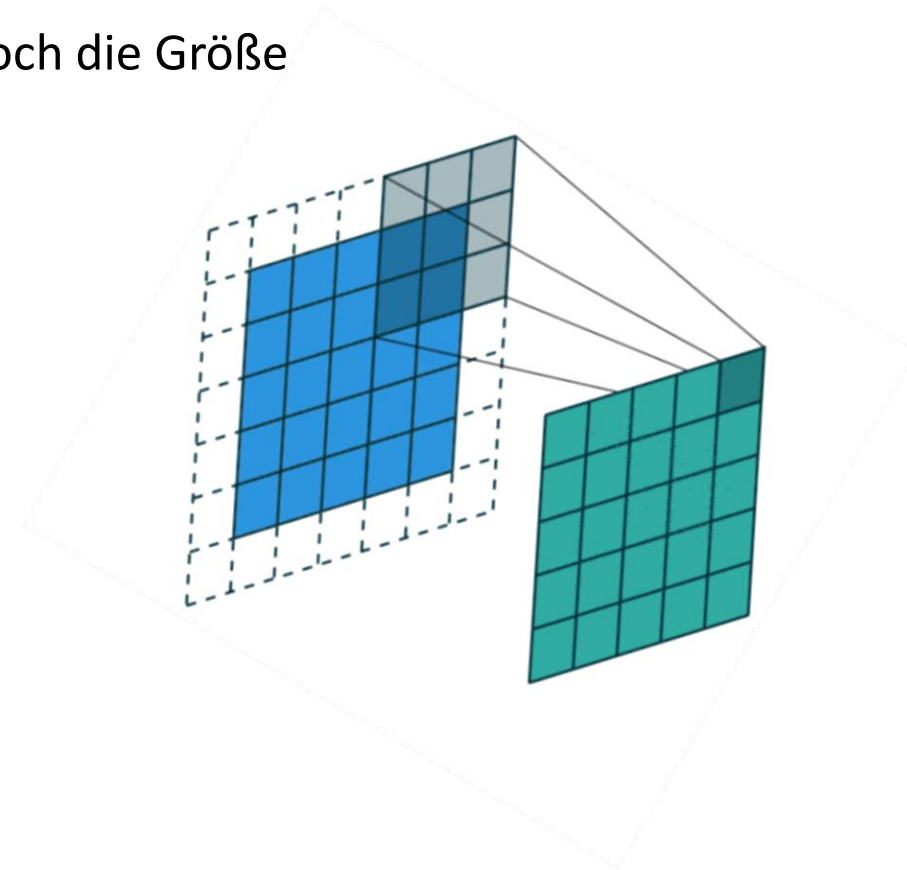
für $1 \leq i, j \leq (\dim(Quelle) - \dim(Filter) + 1)$

Diese Formel entspricht der mathematischen Faltung (engl. **convolution**)

die (mathematische) Faltung (2)

Anschaulich:

- ein Filter, der deutlich kleiner als das Quellbild ist, wird schrittweise Pixel für Pixel über das Quellbild geführt
- es ergibt zudem eine Dimensionsreduktion, da das Abbild nur noch die Größe $\dim(Quelle) - \dim(Filter) + 1$ hat.



die (mathematische) Faltung (2)

Filter werden auch Kernel genannt

Für das folgende Beispiel verwenden wir den Filter $F = \begin{pmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{pmatrix}$

0	0	0	0	0	0	0	0
0	1	0	1	2	3	0	0
0	0	0	1	2	3	0	0
0	0	0	1	2	3	0	0
0	0	0	0	1	2	0	0
0	0	0	1	2	2	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

Quellenbild

1	0	-1
0	0	0
-1	0	1

Filter

0	1	2	2	-2	-3
0	1	0	0	0	0
0	-1	-1	0	1	1
0	0	0	-1	0	1
0	0	-1	-2	1	2
0	-1	-2	-1	2	2

Abbild

Convolutional Neural Network – CNN

CNN = Convolutional Neural Network (auch ConvNets)

Bereits Mitte der 1990er maßgeblich von Yann LeCun entwickelt

- vorwärts gerichtetes KNN (es gibt auch CRNNs)
- Vorrangig für Bilderkennung
- Durchbruch 2015 mit dem Beitrag

Szegedy, Christian; Liu, Wei; Jia, Yangqing; Sermanet, Pierre; Reed, Scott; Anguelov, Dragomir; Erhan, Dumitru; Vanhoucke, Vincent; Rabinovich, Andrew:

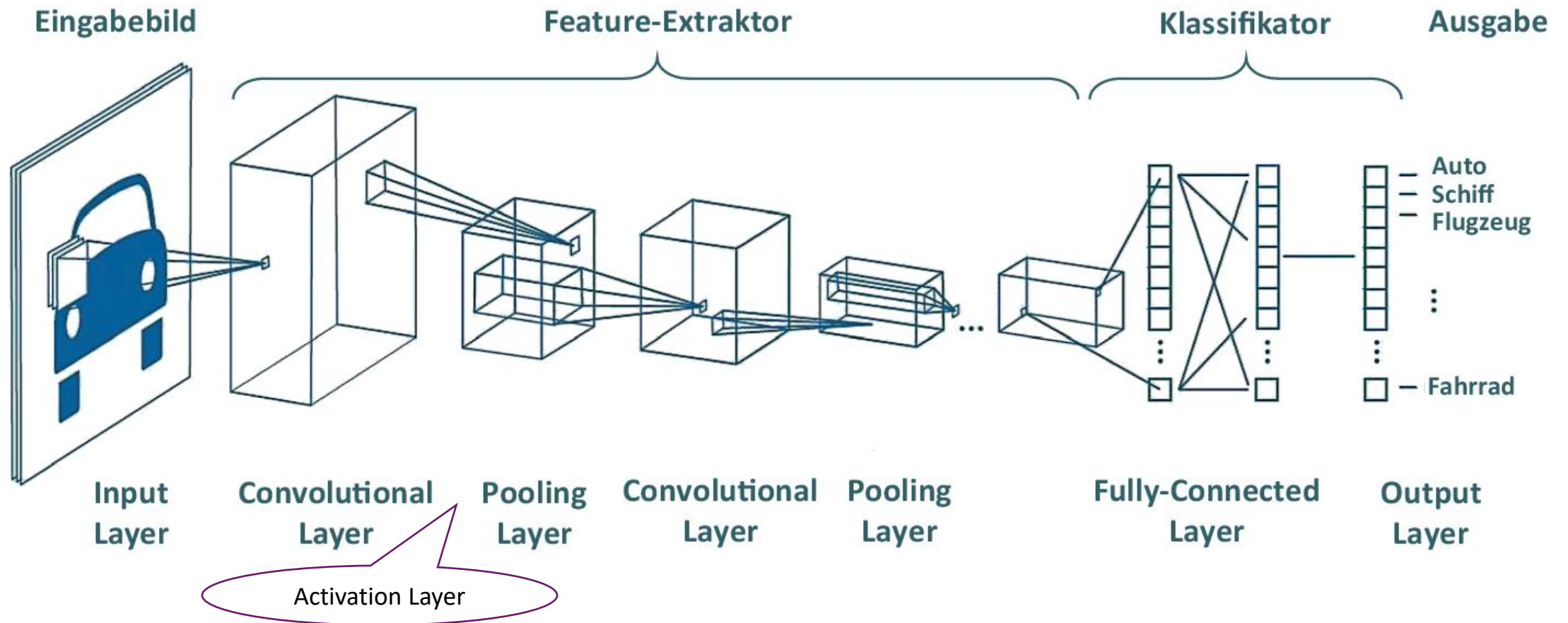
Going deeper with convolutions.

IEEE Conference on Computer Vision and Pattern Recognition, Juni 2015.

Architektur CNN

- Eingabeschicht (*Input Layer*)
- Faltungsschicht (*Convolutional Layer*)
auf die Eingabe werden Filter angewendet, es entsteht eine *Feature Map*
- Aktivierungsschicht (*Activation Layer*)
mit Aktivierungsfunktion, die auf die *Feature Map* angewendet wird
- *Pooling Layer*
reduziert die Dimension der Feature Map, somit werden markanteste Stellen (im Bild) verstärkt
- einem (oder mehreren) *Fully Connected Layer*(s) (auch *Dense Layer*)
- eine Ausgabeschicht (*Output Layer*)

Allgemeiner Aufbau CNN



Bildquelle: MathWorks (2017) Introduction to deep learning: what are convolutional neural networks?

<https://www.mathworks.com/videos/introduction-to-deep-learning-what-are-convolutional-neural-networks--1489512765771.html>

Prinzip CNN

- einfache oder mehrfache Hintereinanderreihung von Convolutional, Activation und Pooling Layer mit einem oder mehreren Fully Connected Layern
- richtige Reihenfolge der Filter, Parameter und Aktivierungsfunktion(en) zu finden, ist die eigentliche Herausforderung (oft viele Versuche notwendig)
- RGB-Bilder werden für die Eingabe in ihre Farbkanäle unterteilt

Convolutional Layer

- Der Convolutional Layer benötigt Filter.
- Die Zahlenwerte für die Filter (auch Kernel genannt) werden zufällig gewählt und während des Trainings angepasst.
- Es gibt bereits spezielle Filter, die bestimmte Effekte bewirken:
 - Sobel-Filter (Kantenhervorhebung)
 - ...

Bei der Anwendung des Filters können schnell sehr große Werte entstehen. Daher werden in der **Praxis** die Bildwerte des Quellenbildes vor der Bearbeitung in den Bereich $[-0.5, +0.5]$ normiert.

Sobel-Filter

Sobel-Filter zur Erkennung vertikale Linien:

-1	0	1
-2	0	2
-1	0	1

Sobel-Filter zur Erkennung horizontale Linien:

1	2	1
0	0	0
-1	-2	-1

Beispiel (1) Pixelcodierung und Sobel-Filter ohne Normierung

7	7	7	7	7	6	6	6	6	6	7	7	7	7	7
7	7	7	6	6	6	6	5	6	5	7	7	7	7	7
7	7	7	6	0	0	5	5	5	5	6	7	7	7	7
7	7	5	2	0	1	6	7	7	6	5	7	7	6	7
7	7	7	5	0	1	7	7	7	7	5	6	6	7	7
7	7	6	6	0	0	7	7	7	1	0	6	6	7	7
7	7	6	6	0	0	6	7	6	1	0	5	5	7	7
7	5	6	6	0	0	6	7	6	0	0	5	6	5	7
7	5	5	5	0	0	5	5	5	0	0	6	5	6	7
7	5	5	5	0	0	1	0	0	1	0	0	5	6	7
7	7	5	5	0	0	0	0	0	0	0	0	0	6	7
7	7	5	5	5	5	3	3	3	0	0	5	5	6	7
7	7	6	5	5	5	5	5	5	0	0	5	6	7	7
7	7	7	6	6	5	5	5	5	0	0	6	7	7	7
7	7	7	7	7	7	6	6	6	0	2	7	7	7	7
7	7	7	7	7	7	7	7	7	7	7	7	7	7	7

-1	0	1
-2	0	2
-1	0	1

0	-3	-9	-7	4	3	0	0	4	7	1	0	0	0
-2	-8	-20	-13	16	15	1	-1	1	7	4	-1	0	1
-4	-13	-24	-12	24	23	2	-2	-5	3	6	-1	1	2
-3	-10	-25	-15	27	25	1	-7	-13	4	10	2	3	1
-3	-5	-25	-22	27	27	0	-18	-22	13	18	5	5	0
-4	-2	-24	-24	25	28	0	-25	-25	18	22	5	6	2
-5	1	-23	-23	23	26	0	-25	-23	20	22	2	6	5
-7	1	-21	-21	17	17	-1	-16	-16	16	21	6	7	5
-8	-2	-20	-20	7	5	-2	-3	-5	4	15	18	13	4
-8	-6	-15	-15	-1	-2	-1	-2	-3	4	10	19	18	4
-7	-8	-6	-5	-4	-4	0	-11	-11	15	16	10	12	3
-4	-7	-3	-1	-3	-2	0	-18	-18	-21	24	6	4	1
-1	-4	-3	-2	-3	-1	0	-21	-19	24	25	4	1	0
0	-1	-1	-1	-3	-2	0	-17	-13	20	17	1	0	0

nach [Lä 2020], S. 255

Beispiel (2) Pixelcodierung und Sobel-Filter mit Normierung

0.5	0.5	0.5	0.5	0.5	0.4	0.4	0.4	0.4	0.4	0.5	0.5	0.5	0.5	0.5	0.5
0.5	0.5	0.5	0.4	0.4	0.4	0.4	0.2	0.4	0.2	0.5	0.5	0.5	0.5	0.5	0.5
0.5	0.5	0.5	0.4	-0.5	-0.5	0.2	0.2	0.2	0.2	0.4	0.5	0.5	0.5	0.5	0.5
0.5	0.5	0.2	-0.2	-0.5	-0.4	0.4	0.5	0.5	0.4	0.2	0.5	0.5	0.4	0.5	0.5
0.5	0.5	0.5	0.2	-0.5	-0.4	0.5	0.5	0.5	0.5	0.2	0.4	0.4	0.5	0.5	0.5
0.5	0.5	0.4	0.4	-0.5	-0.5	0.5	0.5	0.5	-0.4	-0.5	0.4	0.4	0.5	0.5	0.5
0.5	0.5	0.4	0.4	-0.5	-0.5	0.4	0.5	0.4	-0.4	-0.5	0.2	0.2	0.5	0.5	0.5
0.5	0.2	0.4	0.4	-0.5	-0.5	0.4	0.5	0.4	-0.5	-0.5	0.2	0.4	0.2	0.5	0.5
0.5	0.2	0.2	0.2	-0.5	-0.5	0.2	0.2	0.2	-0.5	-0.5	0.4	0.2	0.4	0.5	0.5
0.5	0.2	0.2	0.2	-0.5	-0.5	-0.4	-0.5	-0.5	-0.4	-0.5	-0.5	0.2	0.4	0.5	0.5
0.5	0.5	0.2	0.2	-0.5	-0.5	-0.5	-0.5	-0.5	-0.5	-0.5	-0.5	-0.5	0.4	0.5	0.5
0.5	0.5	0.2	0.2	0.2	0.2	-0.1	-0.1	-0.1	-0.5	-0.5	0.2	0.2	0.4	0.5	0.5
0.5	0.5	0.4	0.2	0.2	0.2	0.2	0.2	0.2	-0.5	-0.5	0.2	0.4	0.5	0.5	0.5
0.5	0.5	0.5	0.4	0.4	0.2	0.2	0.2	0.2	-0.5	-0.5	0.4	0.5	0.5	0.5	0.5
0.5	0.5	0.5	0.5	0.5	0.5	0.4	0.4	0.4	-0.5	-0.2	0.5	0.5	0.5	0.5	0.5
0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5

-1	0	1
-2	0	2
-1	0	1

0.0	-0.4	-1.3	-1.0	0.6	0.4	0.0	0.0	0.6	1.0	0.1	0.0	0.0	0.0	0.0	0.0
-0.3	-1.1	-2.9	-1.9	2.3	2.1	0.1	-0.1	0.1	1.0	0.6	-0.1	0.0	0.1	0.1	0.1
-0.6	-1.9	-3.4	-1.7	3.4	3.3	0.3	-0.3	-0.7	0.4	0.9	-0.1	0.1	0.3	0.3	0.3
-0.4	-1.4	-3.6	-2.1	3.9	3.6	0.1	-1.0	-1.9	0.6	1.4	0.3	0.4	0.1	0.1	0.1
-0.4	-0.7	-3.6	-3.1	3.9	3.9	0.0	-2.6	-3.1	1.9	2.6	0.7	0.7	0.0	0.0	0.0
-0.6	-0.3	-3.4	-3.4	3.6	4.0	0.0	-3.6	-3.6	2.6	3.1	0.7	0.9	0.3	0.3	0.3
-0.7	0.1	-3.3	-3.3	3.3	3.7	0.0	-3.6	-3.3	2.9	3.1	0.3	0.9	0.7	0.7	0.7
-1.0	0.1	-3.0	-3.0	2.4	2.4	-0.1	-2.3	-2.3	2.3	3.0	0.9	1.0	0.7	0.7	0.7
-1.1	-0.3	-2.9	-2.9	1.0	0.7	-0.3	-0.4	-0.7	0.6	2.1	2.6	1.9	0.6	0.6	0.6
-1.1	-0.9	-2.1	-2.1	-0.1	-0.3	-0.1	-0.3	-0.4	0.6	1.4	2.7	2.6	0.6	0.6	0.6
-1.0	-1.1	-0.9	-0.7	-0.6	-0.6	0.0	-1.6	-1.6	2.1	2.3	1.4	1.7	0.4	0.4	0.4
-0.6	-1.0	-0.4	-0.1	-0.4	-0.3	0.0	-2.6	-2.6	3.0	3.4	0.9	0.6	0.1	0.1	0.1
-0.1	-0.6	-0.4	-0.3	-0.4	-0.1	0.0	-3.0	-2.7	3.4	3.6	0.6	0.1	0.0	0.0	0.0
0.0	-0.1	-0.1	-0.1	-0.4	-0.3	0.0	-2.4	-1.9	2.9	2.4	0.1	0.0	0.0	0.0	0.0

Beispiel (3) Pixelcodierung und Sobel-Filter mit Normierung eingefärbt

0,5	0,5	0,5	0,5	0,5	0,4	0,4	0,4	0,4	0,4	0,5	0,5	0,5	0,5	0,5	0,5
0,5	0,5	0,5	0,4	0,4	0,4	0,4	0,2	0,4	0,2	0,5	0,5	0,5	0,5	0,5	0,5
0,5	0,5	0,5	0,4	-0,5	-0,5	0,2	0,2	0,2	0,2	0,4	0,5	0,5	0,5	0,5	0,5
0,5	0,5	0,2	-0,2	-0,5	-0,4	0,4	0,5	0,5	0,4	0,2	0,5	0,5	0,4	0,5	0,5
0,5	0,5	0,5	0,2	-0,5	-0,4	0,5	0,5	0,5	0,5	0,2	0,4	0,4	0,5	0,5	0,5
0,5	0,5	0,4	0,4	-0,5	-0,5	0,5	0,5	0,5	-0,4	-0,5	0,4	0,4	0,5	0,5	0,5
0,5	0,5	0,4	0,4	-0,5	-0,5	0,4	0,5	0,4	-0,4	-0,5	0,2	0,2	0,5	0,5	0,5
0,5	0,2	0,4	0,4	-0,5	-0,5	0,4	0,5	0,4	-0,5	-0,5	0,2	0,4	0,2	0,5	0,5
0,5	0,2	0,2	0,2	-0,5	-0,5	0,2	0,2	0,2	-0,5	-0,5	0,4	0,2	0,4	0,5	0,5
0,5	0,2	0,2	0,2	-0,5	-0,5	-0,4	-0,5	-0,5	-0,4	-0,5	-0,5	0,2	0,4	0,5	0,5
0,5	0,5	0,2	0,2	-0,5	-0,5	-0,5	-0,5	-0,5	-0,5	-0,5	-0,5	-0,5	0,4	0,5	0,5
0,5	0,5	0,2	0,2	0,2	0,2	-0,1	-0,1	-0,1	-0,5	-0,5	0,2	0,2	0,4	0,5	0,5
0,5	0,5	0,4	0,2	0,2	0,2	0,2	0,2	0,2	-0,5	-0,5	0,2	0,4	0,5	0,5	0,5
0,5	0,5	0,5	0,4	0,4	0,2	0,2	0,2	0,2	-0,5	-0,5	0,4	0,5	0,5	0,5	0,5
0,5	0,5	0,5	0,5	0,5	0,5	0,4	0,4	0,4	-0,5	-0,2	0,5	0,5	0,5	0,5	0,5
0,5	0,5	0,5	0,5	0,5	0,5	0,5	0,5	0,5	0,5	0,5	0,5	0,5	0,5	0,5	0,5

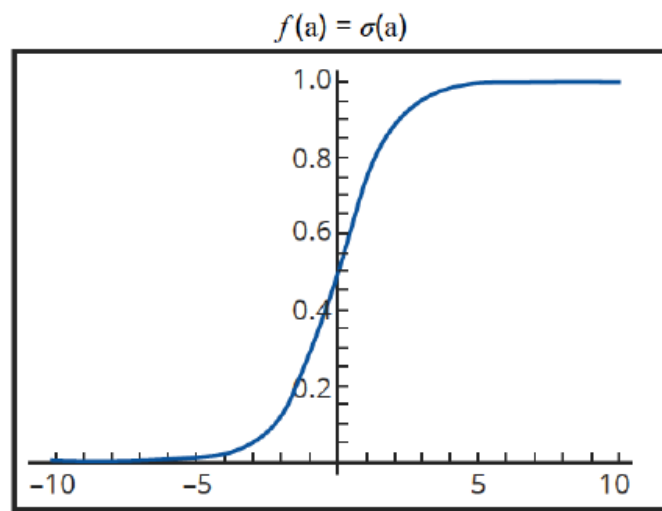
-1	0	1
-2	0	2
-1	0	1

0,0	-0,4	-1,3	-1,0	0,6	0,4	0,0	0,0	0,6	1,0	0,1	0,0	0,0	0,0	0,0	0,0
-0,3	-1,1	-2,9	-1,9	2,3	2,1	0,1	-0,1	0,1	1,0	0,6	-0,1	0,0	0,1	0,1	0,1
-0,6	-1,9	-3,4	-1,7	3,4	3,3	0,3	-0,3	-0,7	0,4	0,9	-0,1	0,1	0,3	0,3	0,3
-0,4	-1,4	-3,6	-2,1	3,9	3,6	0,1	-1,0	-1,9	0,6	1,4	0,3	0,4	0,1	0,1	0,1
-0,4	-0,7	-3,6	-3,1	3,9	3,9	0,0	-2,6	-3,1	1,9	2,6	0,7	0,7	0,0	0,0	0,0
-0,6	-0,3	-3,4	-3,4	3,6	4,0	0,0	-3,6	-3,6	2,6	3,1	0,7	0,9	0,3	0,3	0,3
-0,7	0,1	-3,3	-3,3	3,3	3,7	0,0	-3,6	-3,3	2,9	3,1	0,3	0,9	0,7	0,7	0,7
-1,0	0,1	-3,0	-3,0	2,4	2,4	-0,1	-2,3	-2,3	2,3	3,0	0,9	1,0	0,7	0,7	0,7
-1,1	-0,3	-2,9	-2,9	1,0	0,7	-0,3	-0,4	-0,7	0,6	2,1	2,6	1,9	0,6	0,6	0,6
-1,1	-0,9	-2,1	-2,1	-0,1	-0,3	-0,1	-0,3	-0,4	0,6	1,4	2,7	2,6	0,6	0,6	0,6
-1,0	-1,1	-0,9	-0,7	-0,6	-0,6	0,0	-1,6	-1,6	2,1	2,3	1,4	1,7	0,4	0,4	0,4
-0,6	-1,0	-0,4	-0,1	-0,4	-0,3	0,0	-2,6	-2,6	3,0	3,4	0,9	0,6	0,1	0,1	0,1
-0,1	-0,6	-0,4	-0,3	-0,4	-0,1	0,0	-3,0	-2,7	3,4	3,6	0,6	0,1	0,0	0,0	0,0
0,0	-0,1	-0,1	-0,1	-0,4	-0,3	0,0	-2,4	-1,9	2,9	2,4	0,1	0,0	0,0	0,0	0,0

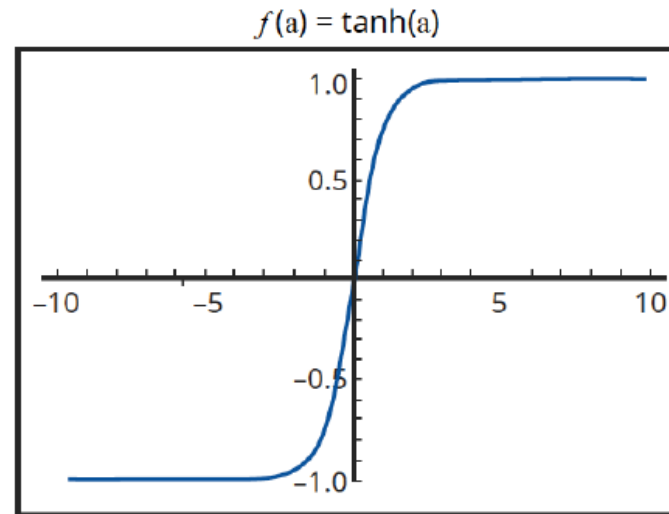
Activation Layer

Benötigt wird eine Aktivierungsfunktion

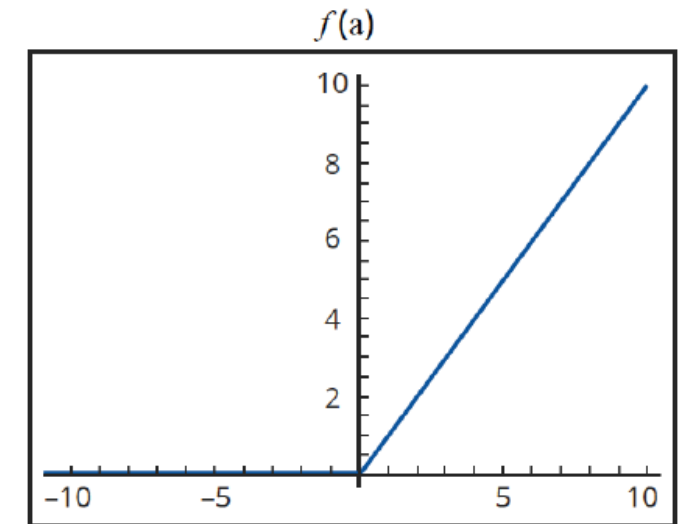
Beispiele:



Sigmoid



Tangens hyperbolicus



Rectified Linear Unit (ReLU)

Beispiel (4) normierte Pixelcodierung, Sobel-Filter und ReLU

0,0	0,0	0,0	0,0	0,6	0,4	0,0	0,0	0,6	1,0	0,1	0,0	0,0	0,0
0,0	0,0	0,0	0,0	2,3	2,1	0,1	0,0	0,1	1,0	0,6	0,0	0,0	0,1
0,0	0,0	0,0	0,0	3,4	3,3	0,3	0,0	0,0	0,4	0,9	0,0	0,1	0,3
0,0	0,0	0,0	0,0	3,9	3,6	0,1	0,0	0,0	0,6	1,4	0,3	0,4	0,1
0,0	0,0	0,0	0,0	3,9	3,9	0,0	0,0	0,0	1,9	2,6	0,7	0,7	0,0
0,0	0,0	0,0	0,0	3,6	4,0	0,0	0,0	0,0	2,6	3,1	0,7	0,9	0,3
0,0	0,0	0,0	0,0	3,3	3,7	0,0	0,0	0,0	2,9	3,1	0,3	0,9	0,7
0,0	0,0	0,0	0,0	2,4	2,4	0,0	0,0	0,0	2,3	3,0	0,9	1,0	0,7
0,0	0,0	0,0	0,0	1,0	0,7	0,0	0,0	0,0	0,6	2,1	2,6	1,9	0,6
0,0	0,0	0,0	0,0	0,0	0,0	0,0	0,0	0,0	0,6	1,4	2,7	2,6	0,6
0,0	0,0	0,0	0,0	0,0	0,0	0,0	0,0	0,0	2,1	2,3	1,4	1,7	0,4
0,0	0,0	0,0	0,0	0,0	0,0	0,0	0,0	0,0	3,0	3,4	0,9	0,6	0,1
0,0	0,0	0,0	0,0	0,0	0,0	0,0	0,0	0,0	3,4	3,6	0,6	0,1	0,0
0,0	0,0	0,0	0,0	0,0	0,0	0,0	0,0	0,0	2,9	2,4	0,1	0,0	0,0

nach [Lä 2020], S. 256

Pooling Layer

Ein Pooling Layer hat die Aufgaben:

- Dimensionsreduktion und damit Reduzierung des Rechenaufwandes
- Unterdrückung des Overfittings
- Erhöhung der translationalen Invarianz

Verfahren:

- MaxPooling (auch Max-Pooling)
- MeanPooling (auch Mean-Pooling)

MaxPooling

Im nachfolgenden Beispiel wird ein MaxPooling-Filter der Größe 3×3 angewendet.

0	0	2	2	-2	-3
0	1	3	0	0	0
0	0	-1	0	1	1
0	0	0	-1	0	1
0	0	-1	-2	1	2
0	-1	-2	-1	2	2

Eingabe

Anwendung von MaxPooling 3×3
und Schrittweite von 3



3	2
0	2

Ausgabe

Fully Connected Layer

Nach dem (letzten) Pooling Layer ist im Normalfall ein vollvernetztes vorwärtsgerichtetes (feedforward) KNN als Fully Connected Layer angehängt, welches die eigentliche Klassifikation durchführt.

- In der Regel besteht dieses KNN aus ein oder zwei hidden Layern.
- Die Größe der Ausgabeschicht richtet sich nach der Anzahl der Klassen, die das Netz erkennen soll.
- Die Vorhersage einer Klasse zur aktuellen Eingabe erfolgt nach dem Softmax-Prinzip (Softmax-Funktion, normalisierte Exponentialfunktion).
- Somit sind prozentuale Aussagen über die Zugehörigkeit zu den Klassen möglich.
- Vorhergesagt wird die Klasse mit der höchsten normierten Aktivität.

Softmax-Funktion anhand eines Beispiels

$$\sigma_j(z) = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}} \quad (1.6)$$

mit z = Vektor der Klassenzuordnung und K Anzahl der Klassen.

Idee ist, dass die Summe der Ausgabe aller Neuronen der Ausgabeschicht 1 ergeben soll.

Beispiel mit 3 Klassen:

$$z = \begin{pmatrix} 0,7 \\ 0,3 \\ 0,5 \end{pmatrix} \quad \sum_{k=1}^3 e^{z_k} = e^{0,7} + e^{0,3} + e^{0,5} = 5,0123$$

$$\sigma_1(z) = \frac{e^{0,7}}{5,0123} \approx \mathbf{0,4018} \quad \sigma_2(z) = \frac{e^{0,3}}{5,0123} \approx \mathbf{0,2693} \quad \sigma_3(z) = \frac{e^{0,5}}{5,0123} \approx \mathbf{0,3289}$$

Was erwartet Dich in diesem Kapitel?

Competitive Networks

Feedforward versus Rekurrent

Rekurrente KNN

Deep Learning – was ist das?

CNN

GAN

Generative Adversarial Networks – GAN (1)

GAN sind Modelle, die neue Daten aus Trainingsdaten generieren können

Prinzip:

2 konkurrierende KNNs treten gegeneinander an

- ein Generator, erzeugt auf Basis der zugrundeliegenden Daten neue fiktive Daten(instanzen)
- ein Diskriminator, der die generierten Dateninstanzen auf „Echtheit prüft“

Verdeutlicht:

- Kunstfälscher*in malt immer wieder neue Gemälde,
- Kunstexpert*in überprüft Gemälde auf Echtheit und muss Fälschungen erkennen
- Sowohl Kunstfälscher*in als auch Kunstexpert*in lernen von den Methoden des/der anderen.

Generative Adversarial Networks – GAN (2)

Wie können Generator und Diskriminator voneinander „lernen“?

Der Vorhersageerfolg des Diskriminators wird als **Fehlergradient** für den Generator verwendet.

Auf diese Weise soll der Diskriminator besser werden, echte Daten von generierten Daten zu unterscheiden, während der Generator lernt, weniger vorhersehbar zu werden.

Dieses dynamische Zusammenspiel kann als eine Art Turing-Test oder als neuronales Korrelat des Minimax-Algorithmus (siehe Kapitel 05_02_Spielen) betrachtet werden.

Der Lern-Prozess ist relativ schwer auszubalancieren, da er nicht konvergiert, wenn entweder der Generator oder der Diskriminator bei seiner jeweiligen Aufgabe zu erfolgreich ist.

Weiterführende Literatur:

Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, Yoshua Bengio:
Generative Adversarial Networks. 2014
<https://arxiv.org/abs/1406.2661>

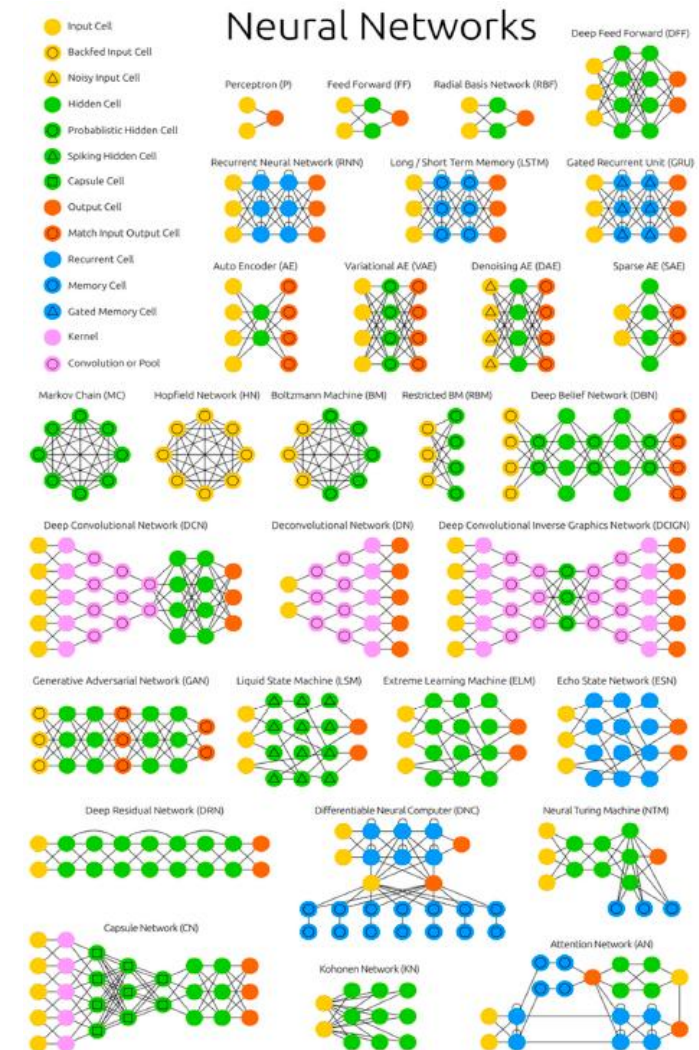
Bekannte Netze

- LeNet (LeNet-5)
1998 von Yann LeCun zur Handschriftenklassifikation entwickelt
<http://yann.lecun.com/exdb/lenet/>
- AlexNet
Bilderkennung
- VGGNet (VGG - Visual Geometry Group): VGG-16 und VGG-19
Klassifizierung von Objektkategorien
in Keras integriert
- YOLO (You only look once)
in Echtzeit Bilder erkennen
- PoseNet
in Echtzeit Stellungen und Bewegungen eines oder mehrerer Menschen im Bild oder Video erkennen

Weitere Netzarchitekturen

Es gibt weitere Netzarchitekturen, die nur für bestimmte Aufgabenstellungen und/oder Fachgebiete entwickelt wurden.

Stefan Leijnen und Fjodor Van Veen listen den *Neural Zoo* auf:
www.researchgate.net/publication/341373030 The Neural Network Zoo



Verwendete Quellen und weiterführende Literatur

[Fr 2018] Frochte, Jörg:
Maschinelles Lernen: Grundlagen und Algorithmen in Python.
Hanser, 1. Auflage, 2018.

[Lä 2020] Lämmel, Uwe; Cleve, Jürgen:
Künstliche Intelligenz.
Hanser, 5. Auflage, 2020.

[Ra 2018] Raschka, Sebastian; Mirjalili, Vahid:
Machine Learning with Python und Scikit-learn und Tensorflow.
mitp, 2, Auflage, 2018.

Otte, Ralf:
Künstliche Intelligenz für dummies.
1. Auflage, Wiley, 2019, S.182

weiterführende Literatur

Chung, Junyoung; Gülçehre, Çağlar; Cho, KyungHyun; Bengio, Yoshua:
[Empirical evaluation of Gated Recurrent Neural Network on Sequence Modeling.](#)
arXiv:1412.3555v1 [cs.NE], 11. Dezember 2014

Hochreiter, Sepp; Schmidhuber, Johannes:
[Long Short-term Memory.](#)
Neural Computation, 9(8), Dezember 1997, S. 1735-1780

Jozefowicz, Rafal; Zaremba, Wojciech; Sutskever, Ilya:
[An empirical exploration of recurrent network architectures.](#)
ICML'15: Proceedings of the 32nd International Conference on International Conference on Machine Learning,
Volume 37, Juli 2015, S. 2342–2350

Pascanu, Razvan; Mikolov, Tomas; Bengio, Yoshua:
On the difficulty of training Recurrent Neural Networks.
arXiv.1211.5063v2 [cs.LG], 16 Februar 2013

Werbos, Paul:
[Backpropagation through time: what it does and how to do it.](#)
Proceedings of the IEEE 78(19), November 1990, S. 1550-1560

Mathematische Videos von E. Weitz

Sobel-Filter:

<https://www.youtube.com/watch?v=W7OpxFbrD84>

Faltung:

<https://www.youtube.com/watch?v=9COnNMknLI4&list=PLb0zKSynM2PCmp5J5LWM3PcZXBaCoQkXj&index=23>